



PUBLIC

2026-04-29

Developer Guide: From Classic ABAP to ABAP Cloud

Content

1	Developer Guide: From Classic ABAP to ABAP Cloud.	3
1.1	Learning About ABAP Cloud.	3
	What is ABAP Cloud?.	4
	Clean Core Extensibility and ABAP-Based Extensions.	14
	ABAP Cloud: Learning Resources	15
1.2	Developing in ABAP Cloud and Consuming Classic APIs.	16
	Reference Scenario.	16
	Integration of BAPI Business Logic in the ABAP RESTful Application Programming Model (RAP): General Considerations.	18
	Exposing a BAPI as OData API Using the ABAP RESTful Application Programming Model.	37
	Adding BAPI Business Logic to a RAP Business Object Using RAP Actions.	55
	Extensibility for BAPI-Based RAP Business Objects.	60
	Using the ABAP Test Cockpit as a Supporting Tool for Adopting ABAP Cloud.	62

1 Developer Guide: From Classic ABAP to ABAP Cloud

This developer guide for transitioning from classic ABAP to ABAP Cloud provides insights and practical steps for development if you want to make your applications cloud-ready while maintaining some legacy code.

Classic ABAP is the ABAP development model for legacy solutions. Classic ABAP is based on ABAP technology without restrictions regarding ABAP language versions, use of development tools or access to repository objects. Classic ABAP is still supported in systems running on SAP S/4HANA and SAP S/4HANA Cloud, private edition.

ABAP Cloud is the ABAP development model for state-of-the-art, cloud-ready, and upgrade-stable business apps, services, and extensions. Compared to classic ABAP, ABAP Cloud uses ABAP technology with the following restrictions:

- The ABAP language version is restricted to the ABAP language version [ABAP for Cloud Development](#).
- Access to other repository objects is restricted to APIs that are released for ABAP for Cloud Development.
- Only ABAP development tools for Eclipse (ADT) can be used.
- Transactional consistency is enforced in ABAP for Cloud Development.

The transactional programming model for ABAP Cloud is the ABAP RESTful Application Programming Model (RAP).

As an experienced ABAP developer, you understand the importance of staying up-to-date with the evolving technological landscape. The shift from classic ABAP to ABAP Cloud is a necessary transition for customers aiming to make their products cloud-ready and future stable. ABAP Cloud also offers other advantages: a release contract framework and a public API lifecycle that guarantees upgrade-safety and lifecycle-stable development.

This guide is tailored to assist you in navigating this transition. It provides comprehensive insights into ABAP Cloud concepts, including an extensibility concept to manage the coexistence of standard ABAP and ABAP for Cloud Development in one system. You gain the essential background knowledge needed to effectively bridge the gap between your existing standard ABAP skills and the requirements of ABAP for Cloud Development.

Furthermore, this guide offers a practical, step-by-step approach to transitioning to ABAP Cloud while keeping some of your standard ABAP code, with a special focus on BAPIs as classic APIs. You learn how to redesign some of your custom applications and build extensions on them in ABAP Cloud, following the recommendations for ABAP extensions in the clean core extensibility model.

1.1 Learning About ABAP Cloud

In this section of the developer guide, you get all the background knowledge, concepts, and links to resources to equip you for a transition from classic ABAP to ABAP Cloud.

[What is ABAP Cloud? \[page 4\]](#)

ABAP Cloud is the ABAP development model to build cloud-ready business apps, services, and extensions.

[Clean Core Extensibility and ABAP-Based Extensions \[page 14\]](#)

With development guidelines for SAP S/4HANA Cloud, private edition, and on-premise, you get a concept to manage the coexistence of classic ABAP development and ABAP Cloud in one system.

[ABAP Cloud: Learning Resources \[page 15\]](#)

This chapter gives you an overview of available learning resources for ABAP Cloud development.

1.1.1 What is ABAP Cloud?

ABAP Cloud is the ABAP development model to build cloud-ready business apps, services, and extensions.

ABAP was first designed for developing enterprise business applications. Over time, new concepts, technologies, and statements have been added, but not all are built for cloud technology. Therefore, ABAP needed to clearly define which technologies and languages can support all necessary cloud features. ABAP Cloud helps streamline this process by providing a stable ecosystem for developing complex enterprise apps involving multiple parties and various layers of extensions.

ABAP Cloud provides everything needed to develop lifecycle-stable and cloud-ready business apps, services, and extensions. It is scalable and flexible. With ABAP Cloud, you can build apps that can quickly adapt to changing business requirements.

The ABAP Cloud development model offers you end-to-end support for your development process by building on the ABAP language and the ABAP ecosystem. And it leverages all the advantages of Cloud development, such as built-in extensibility, lifecycle-stable APIs, and a cloud-optimized ABAP language.

For a comprehensive overview of ABAP Cloud and its concepts, see [ABAP Cloud: Background Concepts and Overview](#).

1.1.1.1 From Classic To Cloud

While keeping proven ABAP qualities like integrated lifecycle management, new tools and development concepts were introduced to support customers in their journey towards the cloud:

Evolution of ABAP Development Tools for Eclipse, Core Data Services, and the ABAP RESTful Application Programming Model

The first step was the introduction of the `ABAP Development Tools for Eclipse` as a modern IDE and alternative to the `ABAP Workbench (SE80)`. From a data-model perspective, `Core Data Services (CDS)` were introduced to define and consume semantically rich data models. CDS was then established as the foundation to implement domain-specific transactional, analytical, or integrational use-cases for services and apps.

The `ABAP RESTful Application Programming Model (RAP)` emerged to support the end-to-end development process for transactional services and applications. RAP expanded on the concept of separating the domain-specific implementation from the business service exposure. This architecture blueprint was then adopted as best practice by the other ABAP aspects, for example embedded analytics.

RAP also natively supported many qualities that are the basis for efficient end-to-end and enterprise-scaling development, such as extensibility, documentability, and testability. These integral parts of its development model became the foundation and baseline of the overall ABAP Cloud development model, and they now represent the best practice for all use cases of ABAP Cloud.

Evolution towards the Cloud and Cloud Qualities

For cloud development, upgrade-safety and stability are core paradigms.

To achieve both, and still give customers the possibility to extend SAP applications and development objects, key user extensibility was initially introduced in SAP S/4HANA Cloud as an extensibility option. The release contract framework was scaled and became the foundation for bringing the required cloud qualities together with the data model developments. This established the foundation for lifecycle-stable extensions with Developer extensibility. With RAP, stateless development for OLTP use cases became the basis for cloud development with ABAP.

General Availability of ABAP Cloud

With SAP S/4HANA 2022 SP00, ABAP Cloud and developer extensibility are also available as part of SAP S/4HANA and SAP S/4HANA Cloud, private edition as a development alternative to classic ABAP development. For more information about how extensibility and ABAP Cloud are connected, see [ABAP Extensibility guide](#). 

In 2022, ABAP Cloud was introduced as an umbrella term encompassing all ABAP innovations of the last years to define the standard technologies and best practices for modern and cloud-ready ABAP development.

1.1.1.2 Overview of the Key Concepts in ABAP Cloud

This chapter gives you an overview of the main development concepts in ABAP Cloud.

Stable Separation between SAP-Code and Custom Code

The main goal of ABAP Cloud is to create a stable and clear separation between SAP Code and custom code to enable faster upgrade cycles. In classic ABAP (classic extensibility), SAP code and custom code are intertwined making the custom code susceptible to any change in SAP code. With a clear separation and stable interfaces, the lifecycle stability of custom code is increased and upgrades are smoother and require less or no fixes to custom code. The separation is achieved by using stable released APIs.

Stable SAP APIs

Publicly available APIs, released either locally or remotely, serve as the interface between SAP code and repository objects.

The basis for upgrade safety and lifecycle stability in Cloud upgrade cycles is the clear separation of SAP code and customer code, while maintaining stable access to SAP development objects and other functionality for customers. In ABAP Cloud, this is achieved with an API release framework for the different repository objects

such as, for example, classes or CDS view entities. With the release contract framework, you can allocate release contracts to different repository objects. The release contract determines the technical purpose of the repository objects, for example if it's intended for use as a local API within your system or as a remote API in an integration scenario. The visibility of the API state of a repository object determines in which development environment and language version the release contract is applicable.

They also impose general restrictions on the released development object to avoid incompatible changes in the future. Once a repository object is released with a release contract, you can safely use it in your implementation. Public released APIs, follow a predefined lifecycle and every deprecated API has a defined successor. Developers using released APIs must consider the allowed compatible changes and access the APIs in such a way that errors and interruptions are avoided. For more information about applicable rules, see [Contract Rules for ABAP Released APIs \(ABAP Keyword Documentation\)](#).

You can find an overview of remote and local released APIs on the [SAP Business Accelerator Hub](#).

For more conceptual information about released APIs in ABAP Cloud, see [Public Released APIs \(ABAP Cloud Guide\)](#).

There are different types of released APIs related for clean core. Remote and local APIs with a release contract are fully ABAP Cloud compatible. For finding and exploring other APIs for clean core development (not listed on the SAP accelerator Hub), refer to [Finding and Exploring Classic APIs for Clean Core Development \[page 21\]](#).

Use of Up-To-Date ABAP

In ABAP, every repository object has a language version attribute to define the applicable syntax rules and the set of repository objects that can be used as APIs in the implementation. The ABAP language version defines which ABAP elements can be used in which context.

The language version Standard ABAP contains the full scope of ABAP statements and unrestricted access to all repository objects. As a consequence, custom developments and SAP code can be coupled without a well-defined interface in between. With Standard ABAP, you can use non-released development objects in your custom implementation.

ABAP for Cloud Development and ABAP for Key Users aim to decouple custom developments from SAP code to ensure the upgrade-safety and to use the latest ABAP syntax in the implementation. Only public released SAP APIs (for example released classes and CDS views) are accessible when developing with strict ABAP language versions. For more details about the language versions, refer to [Language Versions FAQs \(SAP blog\)](#).

All this means that a strict ABAP Language version consists of a well-defined part of the ABAP syntax of Standard ABAP that specifically supports the development process in ABAP Cloud. Both language versions only allow access to repository objects of the same software component or repository objects released with the required release contracts. These stability requirements ensure that your development objects remain stable during upgrades.

Use of Newest Technologies and Concepts

ABAP Cloud is the result of a continuous evolution in the ABAP ecosystem and contains the newest innovations available. ABAP Cloud reflects the best practices and the recommended development approach whenever

possible. Some of the main technologies in ABAP Cloud are ABAP Core Data Services for data modelling and embedded analytics, ABAP RESTful application programming model for developing transactional business apps, and ABAP Development Tools for Eclipse as development IDE.

For an overview of technologies, see [Overview of ABAP Cloud Technologies and Tools \[page 7\]](#).

1.1.1.3 Overview of ABAP Cloud Technologies and Tools

All use cases in ABAP Cloud are supported end-to-end. They benefit from the built-in qualities, as well as the use of reusable services and libraries, along with a specialized set of tools for the different use cases and personas.

Table Captions

- **Name:** Name of the programming model aspect as it is used in this guide.
- **Characteristics:** Description of the main characteristics for each programming model aspect.
- **Based On:** Lists the main technologies that are used for domain-specific modeling in the programming model aspect.
- **Implemented With:** Lists the main programming languages that are used to implement the domain-specific logic in each programming model aspect.
- **Implemented For:** Explains which user type an app is intended for. This is important for the authorization considerations in the **Design** phase because the required steps for business roles and authorizations depend on the user type.
 - Business User: Business users are end users who use SAP Fiori applications, as well as applications and services. These users are authorized by being assigned a specific business role.
 - Communication User: Communication users are customer-owned technical users who are assigned to a communication system to enable integration with other solutions.

Note

This table has invisible columns. You can add them with the [Hide/Show Columns](#) option above the table.

Use Case	Characteristics	Based On	Implemented With	Implemented for
Transactional Use Cases (OLTP)	Transactional use cases require apps and services that run operations such as read, create, update, or delete operations on the data set. The architectural separation of concerns between the domain-specific implementation and the business service exposure lets you expose the same data model for analytical and transactional apps and services. The center piece of RAP is the Business Object (RAP BO). RAP BOs consist of the data model implemented with CDS and enriched with transactional behavior using the behavior definition (BDEF) and its behavior implementation. ABAP Cloud enables you to either develop OData-based services for apps for users that contain UI-specific information, or to develop OData-based Web APIs for unspecified clients, or both at the same time. UI scenarios, namely for creating SAP Fiori UIs, are supported using the OData protocol, while integration scenarios are supported with business events, and OData services.	• ABAP RESTful Application Programming Model • Core Data Services	• ABAP/Entity Manipulation Language	• For Apps: Business User • For OData APIs and Remote Events: Communication User

Use Case	Characteristics	Based On	Implemented With	Implemented for
Analytical Use Cases (OLAP)	Analytical use cases involve analyzing and evaluating multidimensional data models to derive real-time data-driven business decisions. The analytical programming model aspect focuses on creating data models to analyze business data in embedded or cross-system setups and to visualize the data in dashboards or as part of apps. Analytical data models are CDS-based. The analytical provider consists of a reusable star or snowflake schema (based on cubes, dimensions, and hierarchies) and scenario-specific analytical projections (analytical queries). ABAP Cloud enables you to develop Information access (InA) protocol based services for multidimensional user apps. The InA services are either consumed in SAP Fiori UIs or by SAP Analytics Cloud.	Core Data Services	Core Data Services	Business User

Use Case	Characteristics	Based On	Implemented With	Implemented for
Integration Use Cases	Integration use cases require the integration of different apps and services to work seamlessly in an end-to-end business process. You can use data integration to exchange data between two or more parties without being part of specific and predefined business process, for example, for analytical use cases. Process integration requirements can range from data exchange across system boundaries, such as, for example, to trigger follow-on actions with events when a value in an app is changed. The integration programming model aspect covers all use cases of application-to-application integration across a range of different protocols and frameworks.	Core Data Services	- ABAP - Core Data Services	Communication User

1.1.1.4 Main Differences between Classic ABAP and ABAP Cloud

This chapter compares the available technologies in ABAP Cloud and classic ABAP development.

ABAP Language

In ABAP Cloud, only a subset of the ABAP language scope is supported to ensure you are using the newest syntax. For more information about the ABAP language in ABAP Cloud, refer to the [Cloud-Optimized ABAP Language](#) chapter in the ABAP Cloud guide.

Feature	ABAP Cloud	Classic ABAP Development
ABAP Language	ABAP for Cloud Development	- ABAP for Cloud Development - Standard ABAP

Defining Authorization and Access

This chapter compares the features and tool set for SAP administrators to set up and to access the relevant ABAP back-end system:

Feature	ABAP Cloud	Classic ABAP Development
Identity & Access Management (IAM)	Using the administration apps on the SAP Fiori launchpad to configure the SAP BTP ABAP environment/SAP S/4HANA Cloud ABAP environment and to provide the business users with the relevant business roles and business catalogs	General administration tools, for example transaction PFCG or SU01, for accessing and securing the ABAP development process on the AS ABAP.

Feature	ABAP Cloud	Classic ABAP Development
Development authorizations	In accordance with the developer role defined by SAP, an ABAP developer can, for example, do the following: • Create and release transport requests • Debug in read-only mode • Develop in the customer namespace • No administrative tasks are enabled within ADT	In accordance with the provided permissions defined by the authorization administrator of your company, an ABAP developer can, for example, do the following: • Create and release transport requests • Debug with change authorization • Develop in the customer namespace • Administrative tasks, such as terminating tasks using SM50, are enabled using the integrated SAP GUI within ADT

Integration and Connectivity

This chapter compares the features that are relevant when preparing, managing, and organizing development activities:

Feature	ABAP Cloud	Classic ABAP Development
Client-server communication	• Communication is based on HTTP using an ABAP cloud project • Authentication is browser-based and checked on the basis of the identity provider • Single-Sign-On (SSO) is only supported through an external browser	• Communication is enabled through Remote Function Call (RFC) using an ABAP project • Using Remote Function Calls (RFC) as the interface for communication with the ABAP system • Authentication is checked through the AS ABAP • Single-Sign-On (SSO) is supported

Developing Applications

This chapter compares the elementary features which support ABAP developers when building ABAP applications:

Feature	ABAP Platform Cloud	ABAP Platform
ABAP package interface	Not supported	Supported

Feature	ABAP Platform Cloud	ABAP Platform
Supported ABAP programming models	ABAP RESTful Programming Model	• Home ABAP Programming Model for SAP Fiori • ABAP RESTful Programming Model
ABAP language	ABAP for Cloud Development	• Classic ABAP • ABAP for Cloud Development
ABAP for SAP HANA development	Supported for ABAP-Managed Database Procedures (AMDP)	
SAP objects	All released SAP objects can be used Direct Modification of SAP objects is not supported	All SAP objects can be used Modification of SAP objects is supported (not recommended)
Embedded SAP GUI	Not supported	Supported
Supported UI technologies	SAP Fiori	• SAP Web Dynpro • SAP Dynpro • Floorplan manager tools • SAP Fiori • And others

Working with ABAP Tools

This chapter compares the tools which an ABAP developer can use to build ABAP applications:

Feature	ABAP Platform Cloud	ABAP Platform
Source-based and form-based editors for ABAP Dictionary objects	The same features can be used.	
Transport Organizer	The same features can be used.	
Refactoring	The same features can be used.	
Quick assists	The same features can be used.	
Supported search functions	• ABAP object search • Where-used function	• ABAP object search • ABAP source search • Where-used function
ABAP unit testing	The same features can be used.	
ABAP Test Cockpit	The same features can be used.	

Feature	ABAP Platform Cloud	ABAP Platform
Troubleshooting tools	Debugging is supported for read-only. Therefore, for example, the application of variables is not enabled while checking an ABAP application	Debugging also supports the application of, for example, variables while checking an ABAP application
Documentation capabilities with ABAP Doc	The same features can be used.	
Documentation capabilities for development objects	Knowledge Transfer Documents are available for documenting development objects.	Knowledge Transfer Documents or SAPScript are available.

Note

Using Knowledge Transfer Documents is recommended as best practice whenever possible.

For further details about the different technologies, refer to [ABAP Cloud - Recommended Use Cases and Technologies](#).

1.1.1.5 Language Elements in Standard ABAP and ABAP for Cloud Development

While all language elements are allowed in standard ABAP, the set of usable elements is restricted in the ABAP language version ABAP for Cloud Development.

To get a complete overview of the language elements in their supported versions, see the ABAP keyword documentation: [Language Elements in ABAP Versions](#).

1.1.2 Clean Core Extensibility and ABAP-Based Extensions

With development guidelines for SAP S/4HANA Cloud, private edition, and on-premise, you get a concept to manage the coexistence of classic ABAP development and ABAP Cloud in one system.

Background and Motivation

As of the SAP S/4HANA 2022 release, ABAP Cloud and the Cloud extensibility model can be used not only in the SAP S/4HANA Cloud, public edition, but also in the SAP S/4HANA Cloud, private edition, and on-premise. However, as a private cloud or on-premise customer, you can very likely not switch completely to this pure Cloud extensibility model immediately, for the following reasons:

- Your business relies on existing classic custom ABAP code.

- There are missing SAP APIs for the broad feature scope in private cloud and on-premise to replace classic custom code because public SAP APIs and extension points focus on the scope of SAP S/4HANA, public edition.

Therefore, a feasible compromise is needed for a private cloud or on-premise environment.

Guidelines

Check the guidelines for more information about clean core extensibility, about extending SAP S/4HANA Cloud, private edition, and on-premise with ABAP-based extensions, and about API enablement guidelines:

[Clean core extensibility for SAP S/4HANA Cloud](#)

[Extend SAP S/4HANA in the cloud and on premise with ABAP based extensions](#)

[ABAP Cloud API Enablement Guidelines for SAP S/4HANA Cloud, private edition, and SAP S/4HANA](#)

1.1.3 ABAP Cloud: Learning Resources

This chapter gives you an overview of available learning resources for ABAP Cloud development.

Learning about ABAP Cloud

ABAP Cloud implements the newest innovations in ABAP and integrates a lot of concepts and technologies that are different from classic ABAP development like using ABAP development tools for Eclipse or the ABAP RESTful Application programming model.

Topic	Learning Resources	Further Information/Documentation
ABAP Development Tools for Eclipse	Learning: Install ABAP Development Tools	• Getting Started with ADT (ADT Guide)
ABAP RESTful Application Programming Model	• Build an SAP Fiori app Using the ABAP RESTful Application Programming Model	• ABAP RESTful Application Programming Model (RAP Guide) • Getting Started (RAP Guide): Generating a RAP Business Service with the Generate ABAP Repository Objects Wizard
Core Data Services	Exploring ABAP Core Data Services	• ABAP Data Models (ADM Guide)
ABAP Cloud		• ABAP Cloud: Background Concepts and Overview (ABAP Cloud Guide)

1.2 Developing in ABAP Cloud and Consuming Classic APIs

In this hands-on section, you get step-by-step instructions aimed at transitioning from classic ABAP to ABAP Cloud.

A significant portion of this documentation focuses on the practical implementation of BAPI-based RAP business objects, offering step-by-step instructions for exposing BAPIs as OData APIs in the ABAP RESTful Application Programming Model (RAP). Using a reference scenario, you'll learn to develop RAP business objects (RAP BOs) and integrate BAPI business logic into these RAP BOs. The documentation also covers essential knowledge and prerequisites for consuming classic APIs within RAP BOs, including proper handling of conversions, authorizations, locking, numbering, user messages, and more.

Background and Motivation

We recommend that you create new custom developments and extensions in ABAP Cloud by default. Such an approach follows the ABAP Cloud extensibility model that allows the use of released SAP APIs only. Custom objects are created in language version **ABAP for Cloud Development**. ABAP for Cloud development rules are enforced by syntax and runtime checks.

In many areas of SAP S/4HANA, however, the SAP S/4HANA technologies that are used for SAP or custom applications don't allow extensibility on-stack with ABAP Cloud or side-by-side with BTP ABAP: They're using legacy UI frameworks such as Web Dynpro or legacy frameworks such as BAPIs for business objects. In particular, no released SAP APIs are available for building business object extensions in ABAP Cloud.

When such a released SAP API is missing, you can mitigate the missing released APIs for ABAP Cloud: You build custom wrappers around classic APIs, using standard ABAP as the language version. Then release the wrapper with release contract C1 to be used for your own ABAP Cloud developments.

1.2.1 Reference Scenario

To illustrate how a BAPI can be exposed as OData API using the ABAP RESTful Application Programming Model (RAP), the business scenario of material classifications is used as an end-to-end example.

[Reference Scenario: Material Classification \[page 17\]](#)

As an example of how you can expose a BAPI as OData API using a RAP facade, the BAPI **BAPI_CLASS_CREATE** is used. The BAPI is used as a basis to create a RAP business object for material classification and build an SAP Fiori app.

[Downloading the Reference Scenario for BAPI Consumption in RAP \[page 17\]](#)

The reference scenario for BAPI consumption in the ABAP RESTful Application Programming Model (RAP) is demo content that you can import into your ABAP system for test purposes.

1.2.1.1 Reference Scenario: Material Classification

As an example of how you can expose a BAPI as OData API using a RAP facade, the BAPI `BAPI_CLASS_CREATE` is used. The BAPI is used as a basis to create a RAP business object for material classification and build an SAP Fiori app.

From SAP GUI Transaction to SAP Fiori App

In systems running on SAP S/4HANA or SAP S/4HANA Cloud private edition, the transaction [Create Class](#) (transaction code CL01) is available for creating classes. The transaction runs in the legacy UI technology SAP GUI.

The aim of this end-to-end example is to show how you can implement an SAP Fiori app for classifications, using the ABAP RESTful Application Programming Model (RAP) and existing BAPIs.

Related BAPIs

The BAPIs `BAPI_CLASS_CREATE`, `BAPI_CLASS_CHANGE`, and `BAPI_CLASS_DELETE` are the standardized programming interfaces that provide access to the business logic for creating, changing, and deleting classes and to the relevant business data for classifications in the system. While the focus in this documentation is on showing how you can expose `BAPI_CLASS_CREATE` as OData API, `BAPI_CLASS_CHANGE` and `BAPI_CLASS_DELETE` are also briefly covered.

1.2.1.2 Downloading the Reference Scenario for BAPI Consumption in RAP

The reference scenario for BAPI consumption in the ABAP RESTful Application Programming Model (RAP) is demo content that you can import into your ABAP system for test purposes.

Context

The reference scenario for BAPI consumption in RAP helps you to get started with development in ABAP Cloud while reusing business logic written in classic ABAP. The reference scenario contains demo content that you can explore to understand how to build your own BAPI-based RAP business objects. The reference content is available on Github: <https://github.com/SAP-samples/abap-platform-nominated-apis-consumption> .

Procedure

- Import all relevant development objects as described in the `README.md` files of the respective branches.

1.2.2 Integration of BAPI Business Logic in the ABAP RESTful Application Programming Model (RAP): General Considerations

Business Application Programming Interfaces (BAPIs) are well-defined APIs that expose data and functionality for remote process integration. This section provides you with background knowledge how it's possible to integrate BAPI logic into RAP.

Why RAP?

In ABAP Cloud, you develop transactional apps and services with RAP. RAP defines the architecture for efficient end-to-end development of intrinsically SAP HANA-optimized OData services (such as Fiori apps) in ABAP Cloud. RAP supports developing transactional Fiori applications as well as publishing Web APIs.

BAPIs in Classic ABAP and in RAP

In classic ABAP, BAPIs are standard SAP interfaces. They play an important role in the technical integration and in the exchange of business data between SAP components, and between SAP and non-SAP components. BAPIs are originally designed for remote communication. They're not intended for user interface interactions or API consumption within business logic. Therefore, and also depending on the specific BAPI in question, using it in a RAP BO might lead to constraints.

However, BAPIs are available for SAP S/4HANA, and they're well documented and tested, so they're the best choice to mitigate for missing released APIs in SAP S/4HANA that can be used for development in ABAP Cloud.

1.2.2.1 BAPI-Based RAP Business Object: Overview of Repository Objects

Before you start, get an overview of how a BAPI-based business object in the ABAP RESTful Application Programming Model (RAP) is built. It requires CDS view entities, behavior definitions, and other ABAP

repository objects for RAP. In addition, you need a custom wrapper class around the BAPI with a release contract C1 for system-internal use.

Must-Know: Managed Transactional Apps in RAP

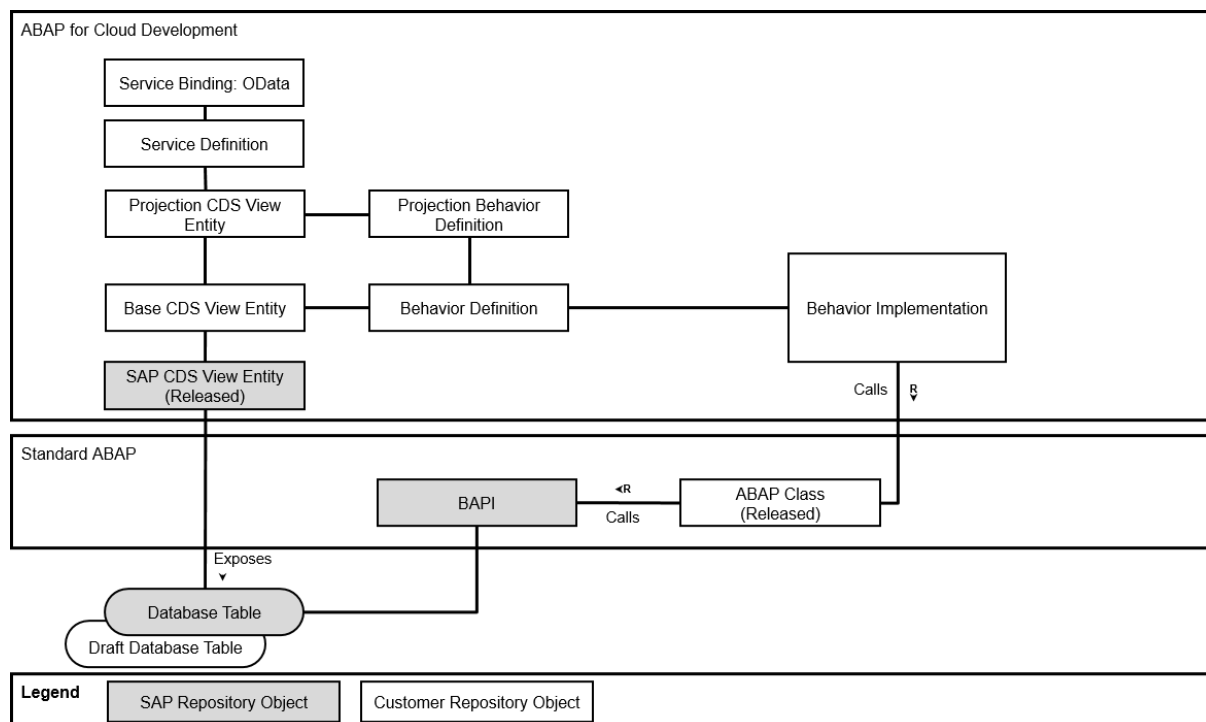
If you aren't familiar with developing transactional apps for SAP Fiori UIs in RAP yet, see [Developing Managed Transactional Apps](#) in the development guide for the ABAP RESTful Application Programming Model (RAP).

Note

If you're eager to learn more about BAPI-based RAP BOs, proceed to the step-by-step instructions in this guide: [Exposing a BAPI as OData API Using the ABAP RESTful Application Programming Model \[page 37\]](#). However, don't forget to come back to the general guidelines and prerequisites before you start creating BAPI-based RAP BOs (see [Consumption of Classic APIs in RAP: Prerequisites and Guidelines \[page 22\]](#)).

BAPI-Based RAP Business Objects

A BAPI-based RAP business object is a RAP business object that consumes business logic of a BAPI by calling it in its behavior implementation. With its CDS data model, the BAPI-based RAP BO retrieves the data from the same persistence as the BAPI.



As a starting point, assume that there's a BAPI that carries out a business task on data stored in a database table whose data is exposed by a released SAP CDS view entity (with release contract C1). If you do not find released CDS view entities for a database table, you can create your own released CDS view entities.

To build a BAPI-based RAP business object, create the following ABAP repository objects:

- CDS view entities on top of existing released CDS view entities, which, in turn, expose the data of the relevant database tables used by the BAPI
- Projections on CDS view entities
- Behavior definition and projection behavior definition to enable CRUD operations for SAP Fiori transactional apps
- A service definition to expose all required CDS view entities and a service binding to expose the service as an OData service

Additionally, to make the BAPI business logic available for consumption in RAP, create an ABAP class and release it with the C1 contract for system-internal use. This class wraps the BAPI written in classic ABAP. The behavior implementation of the business object calls methods from the ABAP wrapper class, which then calls the BAPI.

If you must access artifacts that are not released, you can put them into a behavior definition extension, and this behavior definition extension would then reside in the ABAP language version Standard ABAP. This is a possible approach to consume, for example, non-released authorization objects.

→ Tip

Various generators for building transactional apps in RAP are available (see [OData Web API Service](#)).

1.2.2.2 Packages for Cloud-Ready and C1-Released Development Objects

When you create a BAPI-based business object using the ABAP RESTful Application Programming Model (RAP), consider creating separate packages for ABAP for Cloud development and for development objects written in standard ABAP.

Example

In the reference scenario used in this documentation, the following packages have been created:

- **ZDSAG_CLASIFICATION_CLOUD** for Cloud-ready development objects, with software component **ZABAP_ON_CLOUD** and enforcing **ABAP for Cloud Development** as default ABAP language version. With these settings, development that is ready to be used in the Cloud is enforced. The development objects in this package also comply with clean core principles. This package contains the RAP business object (BO) with its related repository objects, such as CDS view entities, behavior definition, service definition, and ABAP classes for the behavior implementation of the BO.
- **ZDSAG_CLASSIFICATION_C1** for development objects that are created with standard ABAP as default ABAP language version, with software component **HOME**, and intended for release with release contract C1. In such a package, you can release the development objects that you want to consume in the **ZDSAG_CLASIFICATION_CLOUD** package. This package contains the wrappers for BAPIs or function modules and other released artifacts.

Related Information

[Reference Scenario: Material Classification \[page 17\]](#)

[BAPI-Based RAP Business Object: Overview of Repository Objects \[page 18\]](#)

1.2.2.3 Finding and Exploring Classic APIs for Clean Core Development

You can mitigate missing ABAP Cloud APIs for clean core development by building custom wrappers around listed classic APIs using standard ABAP as a language version. There are resources and tools available for finding the appropriate APIs.

List of S/4HANA Cloud Private Edition Classic APIs

The list of available S/4HANA Cloud Private Edition Classic APIs can be downloaded from GitHub: [GitHub - SAP/abap-atc-cr-cv-s4hc](#) .

Getting to Know Selected BAPIs Using the BAPI Explorer

You can also use the BAPI Explorer for getting more details on classic BAPIs. Connect to the back end of your SAP S/4HANA system and start transaction **BAPI**.

BAPI Documentation

All BAPIs offer a function module documentation that provides a detailed description of the input and output parameters. In most cases, examples of standard operations such as **create** are provided. Furthermore, authorization objects are listed and explained.

More Information

[Classic APIs for Tier 2 ABAP Cloud Development in SAP S/4HANA Cloud Private Edition](#) 

1.2.2.4 Finding an ABAP for Cloud Development Successor for a Standard ABAP Repository Object

Find a successor for a standard ABAP repository object in ABAP for Cloud Development.

Context

You want to find a successor in ABAP for Cloud Development for an ABAP repository object from standard ABAP.

For more information, see [Working with ABAP for Cloud Development and Released APIs](#).

Procedure

1. In ABAP Development Tools, search and open the (standard) ABAP repository object in question.
2. Under *Properties*, choose the *API State* tab.
3. Under the C1 contract, you can also find information whether there's a successor object and what it is.

1.2.2.5 Consumption of Classic APIs in RAP: Prerequisites and Guidelines

Business Application Programming Interfaces (BAPIs) are well-defined APIs that expose data and functionality for remote process integration. Depending on the characteristics of a BAPI, it's possible to integrate it into the ABAP RESTful Application Programming Model (RAP).

Consuming BAPIs in RAP

There is no one-size fits all approach when consuming classic APIs such as BAPIs. There's a lot of variety when it comes to consuming a specific BAPI. For example, many BAPIs offer a test run, others don't, or some BAPIs lock the BO when they execute a test run, and so on. However, there are some general guidelines that you should follow when implementing BAPI-based RAP BOs.

Checklist

[BAPI Consumption in RAP: Transactional Consistency \[page 24\]](#)

To consume Business Application Programming Interfaces (BAPIs) in the ABAP RESTful Application Programming Model (RAP), the rules of controlled SAP logical unit of work (LUW) must be adhered to.

[BAPI Consumption in RAP: Test Run and Validations \[page 25\]](#)

Before you choose a BAPI for consumption in the ABAP RESTful Application Programming Model (RAP), check whether the BAPI offers an option for testing its execution.

[BAPI Consumption in RAP: RAP Implementation Type \[page 25\]](#)

If you want to integrate BAPI business logic into a business object in the ABAP RESTful Application Programming Model (RAP), use a managed RAP business object (RAP BO) with unmanaged save.

[BAPI Consumption in RAP: Late Numbering and Save Sequence \[page 26\]](#)

When you create a BAPI-based managed business object (BO) using the ABAP RESTful Application Programming Model (RAP), we recommend that you use late numbering. For the `create` operation, you must consider moving the BAPI call to the `ADJUST_NUMBERS` exit for numbering to work.

[BAPI Consumption in RAP: Feature Control \[page 28\]](#)

To control the visibility and changeability of fields of a BAPI-based RAP business object, use feature control in the ABAP RESTful Application Programming Model (RAP).

[BAPI Consumption in RAP: Messages \[page 29\]](#)

To make BAPI messages consumable in the ABAP RESTful Application Programming Model (RAP), you need a suitable mapping of these messages to the `failed` and `reported` structures in a RAP handler method.

[BAPI Consumption in RAP: Mapping BAPI Structures to CDS Entity Properties \[page 31\]](#)

To make BAPIs consumable in the ABAP RESTful Application Programming Model (RAP), you must find a way to map the BAPI structures to CDS entity properties. You can define this mapping as part of an ABAP class that wraps the BAPI for later consumption in RAP. Alternatively, you can use RAP type and control mapping.

[BAPI Consumption in RAP: Conversions \[page 32\]](#)

As part of a C1-released wrapper class for BAPIs, you must also take care of mappings and conversions between internal and external data formats in a BAPI.

[BAPI Consumption in RAP: Authorizations \[page 33\]](#)

For a BAPI-based business object in the ABAP RESTful Application Programming Model (RAP), you might want to reuse the existing authorization objects of the BAPI. To prevent that a BAPI-based RAP BO performs authorization checks during validation or late save phases where it's not allowed, you define an authorization context in the behavior definition of the RAP BO.

[BAPI Consumption in RAP: Locking \[page 36\]](#)

When you integrate BAPI logic in RAP business objects, you also need a locking mechanism. Since BAPIs come with their own locking mechanisms, it makes sense to reuse the relevant function modules.

[BAPI Consumption in RAP: Data Elements for Draft Database Tables \[page 37\]](#)

For BAPI-based business objects in the ABAP RESTful Application Programming Model (RAP), make sure that you use released data elements or resort to CDS simple types for draft database tables.

1.2.2.5.1 BAPI Consumption in RAP: Transactional Consistency

To consume Business Application Programming Interfaces (BAPIs) in the ABAP RESTful Application Programming Model (RAP), the rules of controlled SAP logical unit of work (LUW) must be adhered to.

Must-Know: Transactional Consistency in RAP

Consuming a classic API in RAP must happen in a transactionally consistent way. Classic APIs must adhere to the rules of controlled SAP LUW and may not violate the transactional consistency by performing forbidden statements in the transactional phases.

If you aren't familiar with controlled SAP LUW yet, see [Controlled SAP LUW](#) in the ABAP development guide.

Consuming BAPIs in RAP with Transactional Consistency

BAPIs are originally designed for remote communication, and they weren't intended for UI interactions or API consumption within a business logic, which makes them a challenge to program with transactional consistency.

BAPIs that perform transactional changes are designed as single requests using Remote Function Call (RFC). The request either succeeds or fails, returning success or fail messages accordingly, which means that a single POST API call includes input data validation as well as the update task.

In UI scenarios, such “one-shot” BAPIs fail to adhere to transactional consistency because user-relevant messages must be returned **before** the update task is registered. Otherwise, the user can't correct issues, so every single issue leads to the termination of the SAP LUW. If a user must correct an issue, the data to be persisted must be changed. An already registered update task can only be invalidated by a rollback, which terminates the current SAP LUW. (For more information about the transactional consistency of one-shot BAPIs and UI scenarios, see [LUW and Transactional Phases](#) in the ABAP development guide.)

To comply with SAP LUW rules, many BAPIs have an input validation during a simulation mode (for example, using a `test_run` parameter or a dedicated “check” BAPI). If the validation returns an error message to the user, the flow can revert to the user for corrections. If the data is correct, the post can be triggered. These BAPIs are candidates for consumption in RAP.

A proven approach is to call the BAPI as late as possible in the processing. Only run it in test mode during the interaction and early save phase, and call it in non-test mode in the late save phase.

(Lack of) Mass Enablement

RAP is mass-enabled by default. Ideally, when you consume BAPIs in RAP, these BAPIs must be able to handle multiple instances. Some BAPIs can only handle one instance at a time, so calling them within one SAP LUW for multiple instances fails. Therefore, you must consider mitigations when you implement BAPI-based RAP

business objects. For example, if a BAPI without mass enablement must be called for an additional instance, you could mitigate the situation by reusing the error message that the BAPI throws and by returning it back to the user.

Most of the time, however, SAP Fiori UIs work with a single instance per call. So, SAP Fiori apps don't encounter issues even if the underlying BAPI itself can only handle one instance at a time. For actions, keep the default invocation grouping (isolated) to avoid grouping multiple instances into one change set.

1.2.2.5.2 BAPI Consumption in RAP: Test Run and Validations

Before you choose a BAPI for consumption in the ABAP RESTful Application Programming Model (RAP), check whether the BAPI offers an option for testing its execution.

A test run parameter is essential because it's needed for validations during the early save phase of the RAP runtime. Without a test run parameter, you will be forced to implement complex workarounds for validations. With the test run parameter, you can reuse BAPI business logic for validations by calling the BAPI in simulation mode.

Related Information

[Implementing Validations for BAPI-Based RAP BOs \[page 50\]](#)

[BAPI Consumption in RAP: Transactional Consistency \[page 24\]](#)

1.2.2.5.3 BAPI Consumption in RAP: RAP Implementation Type

If you want to integrate BAPI business logic into a business object in the ABAP RESTful Application Programming Model (RAP), use a managed RAP business object (RAP BO) with unmanaged save.

Must-Know: Implementation Types of RAP BOs

The standard implementation types of RAP BOs are managed and unmanaged. With the managed implementation type of a RAP BO, you can benefit from out-of-the-box support for transactional processing. For example, standard operations (create, update, delete) must only be specified in the behavior definition to obtain a ready-to-run business object. In addition, the provisioning and handling of the transactional buffer is automatically done by RAP.

For more information, see [Business Object Implementation Types](#) in the development guide for the ABAP RESTful Application Programming Model (RAP).

BAPI-Based RAP BO: Managed with Unmanaged Save

For creating a BAPI-based RAP business object (RAP BO), use a managed RAP BO with an unmanaged save. In a RAP BO with managed implementation and unmanaged save, the RAP-managed provider controls the interaction phase. You implement the late save phase yourself and integrate business logic for saving in the saver methods. This implementation type is suitable for a BAPI-based RAP BO. You can use the out-of-the-box RAP support for transactional processing and still incorporate BAPI business logic with the unmanaged save.

Related Information

[BAPI Consumption in RAP: Late Numbering and Save Sequence \[page 26\]](#)

1.2.2.5.4 BAPI Consumption in RAP: Late Numbering and Save Sequence

When you create a BAPI-based managed business object (BO) using the ABAP RESTful Application Programming Model (RAP), we recommend that you use late numbering. For the `create` operation, you must consider moving the BAPI call to the `ADJUST_NUMBERS` exit for numbering to work.

Must-Know: Late Numbering and Save Sequence in the ABAP RESTful Application Programming Model (RAP)

With late numbering, the `ADJUST_NUMBERS` method is called during the save sequence after the point of no return. At this point in time, the transactional buffer is in a consistent state and all new instances can potentially receive their final number. In the RAP BO runtime, the `ADJUST_NUMBERS` method call is followed by saving the current transactional buffer to the database.

If you aren't familiar with `ADJUST_NUMBERS` and the save sequence in RAP yet, see [ADJUST_NUMBERS](#) and [Save Sequence Runtime](#) in the development guide for the ABAP RESTful Application Programming Model (RAP).

Late Numbering and Save Sequence for BAPI-Based RAP BOs

With BAPIs, the final key of a newly created BO instance is only available after the actual BAPI posting has run. Therefore, you must use late numbering in RAP for assigning the ID to the new BAPI-managed RAP BO instance.

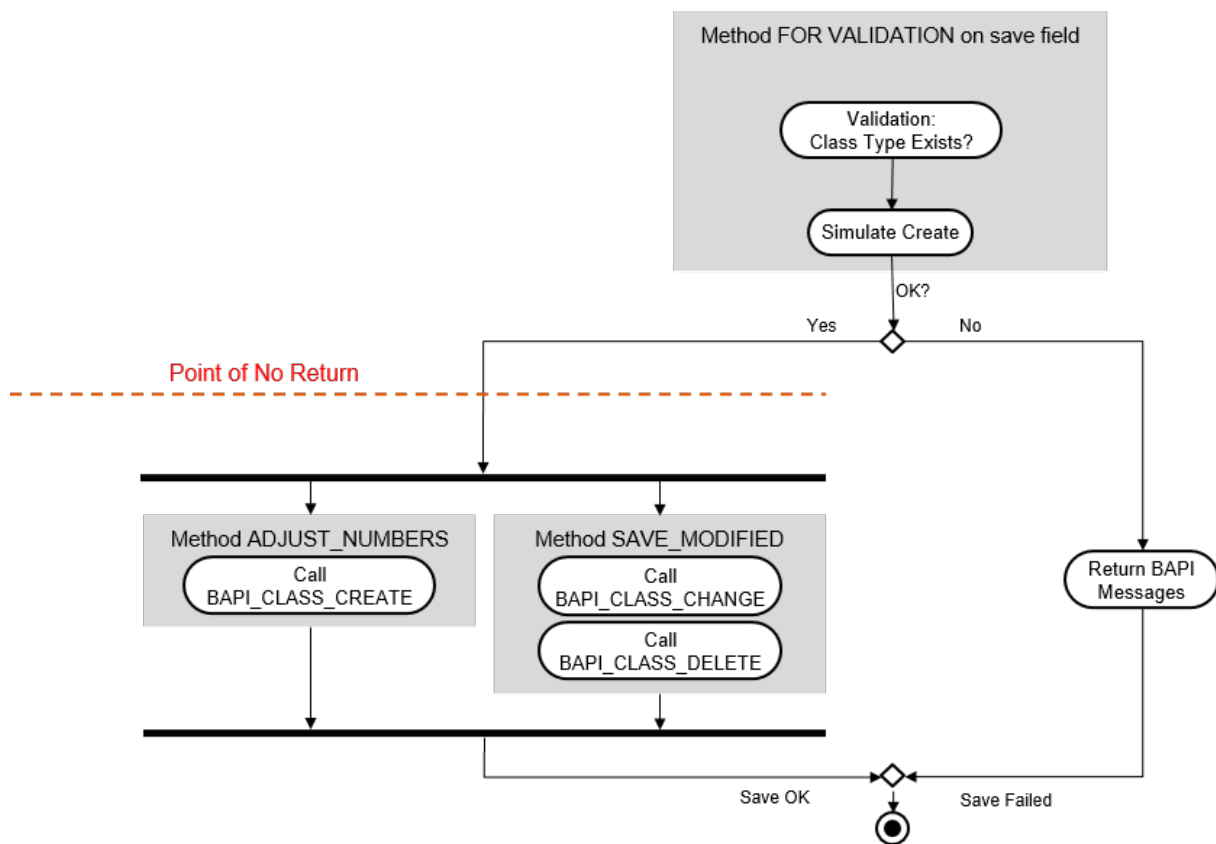
The number is returned as part of the `create` BAPI call. However, in RAP, the calculated number must be returned as part of the `adjust_numbers` exit. As a consequence, you must move the `create` BAPI call from the `save` exit to the `adjust_numbers` exit. The `update` and `delete` BAPI calls can remain in the `save` exit.

To handle BAPI error messages, you must change the inheritance of your local saver class from `cl_abap_behavior_saver` to `cl_abap_behavior_saver_failed`. This change enables the additional parameter `failed` in method `ADJUST_NUMBERS`. BAPIs often use `CALL FUNCTION ... IN UPDATE TASK`, which is forbidden during RAP BO runtime, except in `ADJUST_NUMBERS` and the save methods.

For more information, see [RAP Saver Classes and Methods \(ABAP Keyword Documentation\)](#).

Example

The following flow diagram illustrates the save phases with late numbering for the BAPI-managed RAP business object of the reference scenario:



BAPI-Based, Managed RAP BO with Unmanaged Save and Late Numbering: Flow Diagram

After all requested data records have been put into the RAP transactional buffer using the RAP `create` operation, the OData runtime triggers the RAP save sequence.

During the early save phase, the following validations are performed:

- A validation checks whether the class type *Material* already exists.
- The creation of a new instance is simulated using the BAPI test run, which triggers the return of BAPI error messages if there are any user input errors.

After the point of no return, during the late save phase, the following methods are called:

- The method `ADJUST_NUMBERS` with the call of BAPI `BAPI_CLASS_CREATE` for creating new instances

- The method `SAVE_MODIFIED` with the call of BAPI `BAPI_CLASS_CHANGE` or `BAPI_CLASS_DELETE` for changing or deleting instances

If the BAPI call has been successful, the OData runtime ends the save sequence with a commit.

Related Information

[Implementing BAPI Call, Late Numbering, and Unmanaged Save for BAPI-Based RAP BOs \[page 51\]](#)

1.2.2.5.5 BAPI Consumption in RAP: Feature Control

To control the visibility and changeability of fields of a BAPI-based RAP business object, use feature control in the ABAP RESTful Application Programming Model (RAP).

Must-Know: Feature Control in RAP

In classic ABAP, particularly in BAPIs and in SAP GUI transactions, a common behavior is that specific fields are shown as read-only on the UI by default. For example, after a user has created a class using transaction [Create Class](#) (CL01), the class can still be changed, but the [Class Type](#) field is read-only in the class.

With feature control in RAP, you can provide information to a business service on how data must be displayed for consumption in an SAP Fiori UI, for example, whether fields are mandatory or read-only. If you aren't familiar with feature control yet, see [Feature Control](#) in the development guide for the ABAP RESTful Application Programming Model (RAP).

Adding Feature Control to BAPI-Based RAP Business Objects

For a good user experience, it's important to define feature control in BAPI-based RAP business objects (RAP BOs), too. To take the class example again, if the [Class Type](#) field isn't defined as read-only in the behavior definition of a BAPI-based RAP BO, the field is displayed as editable on the UI. As a result, users might attempt at changing the class type. However, the BAPI used in the back-end ignores the user change without raising an error. The user is falsely led to believe that the update was successful, though it wasn't.

To avoid such issues, feature control logic must be enforced in the RAP behavior definition. Feature control is particularly relevant for update scenarios.

Example

In the reference scenario used in this documentation, the field for the class-internal ID must be defined as read-only, and the field for class type must be defined as mandatory during creation and as read-only during

changes. In the behavior definition of the root entity ZDSAG_R_CLASSIFICATION, you define the feature control as follows:

Sample Code

```
define behavior for ZDSAG_R_CLASSIFICATION alias Classification
" ...
  field ( readonly ) ClassInternalID;
  field ( mandatory : create, readonly : update ) ClassType;
```

With this behavior definition, the field for the class type is read-only when the user tries to update the data.

Additional Considerations for BAPI-Based RAP BOs

While the example of the *Class Type* field is a standard case, don't forget that there could be more complex scenarios. You might have standard or custom extension fields whose changeability is controlled by configuration tables written in classic ABAP. In such cases, if there is no released CDS view entity available for these configuration tables, create a wrapper class. Use logic within the RAP behavior implementation layer for dynamically controlling how fields are displayed (dynamic feature control).

For more information about dynamic feature control, see [Feature Control](#) in the development guide for the ABAP RESTful Application Programming Model (RAP).

1.2.2.5.6 BAPI Consumption in RAP: Messages

To make BAPI messages consumable in the ABAP RESTful Application Programming Model (RAP), you need a suitable mapping of these messages to the `failed` and `reported` structures in a RAP handler method.

Must-Know: Messages in RAP

In RAP, when you execute a modify request, the keys of the failed instances are returned in the `failed` response parameter. Related messages for the failed instance are returned with the `reported` response parameter.

If you aren't familiar with messages in RAP yet, see [Messages](#) in the development guide for the ABAP RESTful Application Programming Model (RAP).

Mapping and Filtering BAPI Messages for RAP

Each BAPI has an export table named `return` of type `BAPIRET2`. This table is used to report any messages, including errors. Whenever the return table contains an error message, this means an unsuccessful execution of the BAPI.

In a BAPI-based RAP business object, you can map these BAPI messages to the **failed** and **reported** structures in a RAP handler method. As a result, the BAPI messages are also displayed on the UI of the transactional app that is based on the RAP BO. A business user can react accordingly to error messages or warnings.

Not all BAPI messages are useful for a business user. For example, the return table might also contain technical messages, like **BAPI executed in simulation mode**. Therefore, you might want to filter such messages when mapping the result to the **reported** structure in a RAP handler method, for example, during a validation.

In RAP, with the **%element** component of the **reported** structure, you can assign messages to one or several target fields. The **BAPIRET2** structure of a BAPI also contains fields for a dedicated field reference of a message. However, these fields are typically not filled, so a mapping to the corresponding **%element** of the **reported** structure in RAP isn't possible. As a workaround, you could manually map the top-x messages to the related **%element** information.

Example

The method `handle_bapi_return` in the behavior pool `zbp_dsag_r_classification` handles errors that a called BAPI returns during the creation or change of a material class. The method filters for errors and warnings, appending the keys of the failed class instances to the **failed-classification** structure. An entry is also added to the **reported-classification** structure with details from the BAPI message, including its severity (error or warning), the message class (**ret-id**), and the message number (**ret-number**). The **%element** component is filled using a workaround: The affected target fields are determined using the message class and message number. For example, if the message number is `'CL 011'`, the **ValidityEndDate** field is marked as having an error.

Sample Code

```
METHOD handle_bapi_return.
  APPEND VALUE #( %tky          = header_tky-%tky
                  %state_area = 'ERROR' )
  TO reported-classification.
  LOOP AT return INTO DATA(ret).
    CASE ret-type.
      WHEN 'E' OR 'W'.
        IF ret-type = 'E'.
          APPEND VALUE #( %tky = header_tky-%tky ) TO failed-classification.
        ENDIF.
        APPEND VALUE #(
          %tky          = header_tky-%tky
          %state_area   = 'ERROR'
          %msg          = new_message(
            id          = ret-id
            number      = ret-number
            severity     = COND #(
              WHEN ret-type = 'E' THEN
if_abap_behv_message=>severity-error
              WHEN ret-type = 'W' THEN
if_abap_behv_message=>severity-warning
            )
            v1          = ret-message_v1
            v2          = ret-message_v2
            v3          = ret-message_v3
            v4          = ret-message_v4 )
          %element-ValidityEndDate = COND #(
            WHEN ret-type = 'E' AND ret-
id = 'CL' AND ret-number = '011'
```

```

                %element-Class = COND #(
id = 'CL' AND ret-number = '002'
                THEN if_abap_behv=>mk-on )
                %element-ValidityStartDate = COND #(
id = 'CL' AND ret-number = '010'
                THEN if_abap_behv=>mk-on ) )
                TO reported-classification.
            WHEN OTHERS.
                " No handling for success or information messages
            ENDCASE.
        ENDLOOP.
    ENDMETHOD.

```

1.2.2.5.7 BAPI Consumption in RAP: Mapping BAPI Structures to CDS Entity Properties

To make BAPIs consumable in the ABAP RESTful Application Programming Model (RAP), you must find a way to map the BAPI structures to CDS entity properties. You can define this mapping as part of an ABAP class that wraps the BAPI for later consumption in RAP. Alternatively, you can use RAP type and control mapping.

Background

When you want to reuse existing BAPI code and data types in the behavior pools of RAP business objects, you must map CDS field names and types to corresponding BAPI fields and types. Different approaches are possible.

Mapping as Part of a Wrapper Class

You create a wrapper class that encapsulates functionality to call BAPIs. The wrapper class also serves to map CDS view entity properties to BAPI structures. This approach is also outlined in this documentation using a reference scenario.

For more information about creating wrapper classes for BAPIs including an example, see [Creating a Wrapper Class for Calling a BAPI \[page 46\]](#).

RAP Type and Control Mapping

As of SAP S/4HANA 2020, you can use RAP type mapping to map BAPI structures to CDS entity properties. Instead of defining the mapping in the wrapper class, you define the mapping as part of the behavior definition of a BAPI-based RAP business object.

In addition, you can use control mapping in RAP. With control mapping in the behavior definition, you map control structures in BAPIs to the `%CONTROL` structure in RAP business objects, which indicates that fields in the main structure are accessed by an operation (update, for example). This control mapping eases the consumption of update BAPIs in RAP business objects: In the behavior implementation of the RAP business object, you can use a variant of the `CORRESPONDING` operator to map control flags to the BAPI change flags before calling the BAPI in an update scenario.

With a central, declarative mapping within the behavior definition instead of many individual corresponding statements in your code, you improve the maintainability of the application source code. Note that this is optional, especially for releases lower than SAP S/4HANA 2025. In releases below 2025, the `%control` information is not available in the validation exits via an addition to the EML `READ` statement. Therefore, in these releases, you can't benefit from the control mapping and must calculate update information manually.

For more information about control structures in RAP, see [Implicit Response Parameters](#) in the ABAP RESTful Application Programming Model documentation. For more information about RAP type and control mapping, see [Using Type and Control Mapping](#) in the ABAP RESTful Application Programming Model documentation. For more information about the `CORRESPONDING` operator, see [CORRESPONDING, Type Mapping for ABAP EML \(ABAP Keyword Documentation\)](#).

1.2.2.5.8 BAPI Consumption in RAP: Conversions

As part of a C1-released wrapper class for BAPIs, you must also take care of mappings and conversions between internal and external data formats in a BAPI.

Internal and External Formats in BAPIs and Implications for BAPI Consumption in the ABAP RESTful Application Programming Model (RAP)

To meet requirements from various applications, fields in BAPI parameters are usually represented in the internal form used in the database and not in a form formatted in the BAPI interface. However, there are exceptions where the calling program has to map and convert the data between the internal and external data formats in a BAPI. These exceptions where mapping is required include currency amounts, quantities with units of measurement, and ISO codes.

In classic ABAP, you can use function modules that can map and, if necessary, convert between the internal and external data formats. When you add BAPI business logic to a RAP business object, you must also take care of these conversions from internal to external formats and conversely. Alpha conversions that add leading zeros if the character field only contains numbers are probably not critical. However, consider the following:

- Standard code lists (for example, the ISO code for currencies) or specific formats for IDs (for example, GTIN for products): If the internal representation isn't part of the BAPI signature, you must map external and internal format accordingly, for example, using mapping function modules in function group `ISOC`.
- Quantities with units of measurement and amount fields (data type `CURR`): For example, for amount fields, the ABAP data type might be handled without a fixed point (that is, the decimal point is irrelevant) and the number of decimals is derived from the referenced currency code. Mapping the values properly is essential and there are related mapping function modules in function group `BFCR`.

To be able to include such conversions into your ABAP Cloud code, include the relevant function modules for conversion in a C1-released wrapper class for the BAPI and call the function modules using the wrapper class in your implementation.

Example

In the reference scenario used in this documentation, language codes for the material class description must be converted from the SAP-internal language code to the ISO language code. This conversion is included in the C1-released wrapper class `zdsag_bapi_class_create` in method `map_4_bapi_class_create`, which handles the conversions for the material classification BAPIs. The method calls function module `LANGUAGE_CODE_SAP_TO_ISO`:

Sample Code

```
CLASS zdsag_bapi_class_create
"
...
METHOD map_4_bapi_class_create.
"
...
  LOOP AT it_descriptions ASSIGNING FIELD-SYMBOL(<fs_description>).
    ls_description-langu = <fs_description>-LanguageEdit.
    CALL FUNCTION 'LANGUAGE_CODE_SAP_TO_ISO'
  ##UNHANDLED_FUNC_EXCEPTION_OK
    EXPORTING
      sap_code = ls_description-langu
    IMPORTING
      iso_code = ls_description-langu_iso.
    ls_description-catchword = <fs_description>-ClassDescription.
    APPEND ls_description TO lt_descriptions.
    CLEAR ls_description.
  ENDLOOP.
```

Related Information

[Creating a Wrapper Class for Calling a BAPI \[page 46\]](#)

1.2.2.5.9 BAPI Consumption in RAP: Authorizations

For a BAPI-based business object in the ABAP RESTful Application Programming Model (RAP), you might want to reuse the existing authorization objects of the BAPI. To prevent that a BAPI-based RAP BO performs authorization checks during validation or late save phases where it's not allowed, you define an authorization context in the behavior definition of the RAP BO.

Must-Know: Defining Authorizations in RAP and Strict Mode

For BAPI-based RAP BOs, you must also define authorizations. Since most of the BAPIs have corresponding authorization objects, these authorization objects of the BAPI are the natural candidates for authorization checks.

In RAP, it's best practice to enforce `strict(2)` mode to ensure that a RAP business object is lifecycle-stable and upgrade-safe. In particular, authorization checks aren't allowed during early or late save phases in RAP (for example, during validations). As a consequence, a BAPI consumption in a RAP BO runs into runtime errors whenever a BAPI is called in simulation mode and performs an authorization check during a validation or determination.

RAP provides a solution for such situations: the authorization context for disable, which prevents the authorization objects from being checked during early or late save phases in RAP, for example. For more information, see [Authorization Definition](#) and [Implementing Instance Authorization](#) in the development guide for the ABAP RESTful Application Programming Model (RAP).

Reusing Authorization Objects for BAPI-Based RAP BOs

To define and implement an authorization check that's compliant with RAP best practices while still reusing the BAPI authorization objects, you define an authorization context for disable in the behavior definition for validations, determinations, and late save.

In the behavior implementation, you implement the RAP handler method `get_instance_authorizations`. This method calls a method in a released wrapper class to check user authorizations against the BAPI authorization objects. This authority check is performed during the operations runtime, but not during validations, determinations, or the late save phase in RAP.

Example

In the header of the behavior definition of the BAPI-based RAP BO `ZDSAG_R_CLASSIFICATION`, the authorization context for disable `NoCheckWhenValidate` has been defined for validations, determinations, and the late save phase. As a result, when a RAP call is performed in privileged mode for validations, determinations and in the late save phase, the BAPI authorization check always results with `sy-subrc` as `0` for the authorization objects listed there:

Sample Code

```
managed with unmanaged save implementation in class zbp_dsag_r_classification
unique;
strict ( 2 );
"...
define authorization context NoCheckWhenValidate for disable ( validation,
determination, save : late )
{
  'C_TCLA_BKA';
  'C_KLAH_BKP';
  'C_KLAH_BKL';
}
```

The actual authority check is then performed when the BAPI is called and when it saves the data to the database. This authority check is implemented as follows: In the behavior implementation of the BAPI-based RAP BO ZDSAG_R_CLASSIFICATION, in the RAP handler method `get_instance_authorizations`, the method `is_allowed_to_modify_class` is called:

Sample Code

```
METHOD get_instance_authorizations.
  DATA lv_activity TYPE activ_auth.
  "...
  LOOP AT Classifications ASSIGNING FIELD-SYMBOL(<fs_classification>).
    IF <fs_classification>-Class IS INITIAL.
      lv_activity = '01'.
    ELSE.
      lv_activity = '02'.
    ENDIF.
    DATA(is_allowed) =
zcl_dsag_class_auth_check=>is_allowed_to_modify_class(
      iv_classnumnew = <fs_classification>-Class
      iv_classtypenew = <fs_classification>-classtype
      iv_activity      = lv_activity ).
```

The method `is_allowed_to_modify_class` checks the authorizations against the BAPI authorization objects. The method is part of a released wrapper class `zcl_dsag_class_auth_check`:

Sample Code

```
CLASS zcl_dsag_class_auth_check IMPLEMENTATION.
  METHOD is_allowed_to_modify_class.
    rv_granted = abap_true.
    SELECT SINGLE ClassClassfctnAuthGrp
      INTO @DATA(lv_authgroup)
      FROM I_ClfnclassDEX
      WHERE class = @iv_classnumnew
        AND ClassType = @iv_classtypenew.
    IF lv_authgroup IS INITIAL OR ( iv_classnumnew IS INITIAL AND
iv_classtypenew IS INITIAL ).
      AUTHORITY-CHECK OBJECT 'C_KLAH_BKL'
                        ID   'BGRKL' DUMMY
                        ID   'ACTVT' FIELD iv_activity.
    ELSE.
      AUTHORITY-CHECK OBJECT 'C_KLAH_BKL'
                        ID   'BGRKL' FIELD lv_authgroup
                        ID   'ACTVT' FIELD iv_activity.
    ENDIF.
    IF sy-subrc <> 0.
      rv_granted = abap_false.
    ENDIF.
    IF iv_classtypenew IS NOT INITIAL.
      AUTHORITY-CHECK OBJECT 'C_TCLA_BKA'
                        ID   'KLART' FIELD iv_classtypenew.
      IF sy-subrc <> 0.
        rv_granted = abap_false.
      ENDIF.
    ENDIF.
  ENDMETHOD.
```

1.2.2.5.10 BAPI Consumption in RAP: Locking

When you integrate BAPI logic in RAP business objects, you also need a locking mechanism. Since BAPIs come with their own locking mechanisms, it makes sense to reuse the relevant function modules.

Must-Know: Locking in the ABAP RESTful Application Programming Model (RAP)

In RAP, concurrency control is handled by the RAP framework out of the box. You only need to implement locking if it's non-standard behavior, using a dedicated method in the behavior pool. For more information, see [<method> FOR LOCK](#).

Locking for BAPI-Based RAP Business Objects

As a locking mechanism for consuming a BAPI in the ABAP RESTful Application Programming Model (RAP), consider reusing the original SAP lock object of the BAPI. To be able to reuse these lock objects, you must create a wrapper class for the enqueue function modules in classic ABAP. Release the wrapper class for internal use (release contract C1).

To implement the use of the BAPI lock object in RAP, use an unmanaged lock handler of your custom RAP BO implementation:

- Define the locking as unmanaged in the behavior definition of the RAP BO root entity.
- In your custom RAP BO implementation, call the relevant function modules for locking via their wrapper class.

The unmanaged lock handler in RAP is required because you must prevent that locking in the RAP context and the SAP GUI context collide. No matter whether a user edits the business object in an SAP Fiori app or in the relevant SAP GUI transaction for the business object, the object must be locked for other users in all UI technologies.

Related Information

[Implementing an Unmanaged Lock for BAPI-Based RAP Business Objects \[page 52\]](#)

1.2.2.5.11 BAPI Consumption in RAP: Data Elements for Draft Database Tables

For BAPI-based business objects in the ABAP RESTful Application Programming Model (RAP), make sure that you use released data elements or resort to CDS simple types for draft database tables.

Must-Know: Draft Database Tables in RAP

As part of a RAP BO, you also need a database table for storing draft instances. RAP lets you generate a draft database table based on the existing database table for the active persistence. For more information about draft database tables in RAP, see [Draft Database Table](#) in the development guide for the ABAP RESTful Application Programming Model (RAP).

Data Elements in Draft Database Tables

In the draft database table, the fields must be typed with released data elements (release contract C1 for internal use). When you generate a draft database table based on the database table for the persistence of your BAPI, you might get error messages if not all fields are typed with release data elements. To address this issue, you can search for appropriate released data elements: Depending on the release of SAP S/4HANA Cloud, private edition, you can find many data elements that are released for internal reuse. For more information about these released data elements, see SAP Note [3470426](#) .

If you can't find appropriate released data elements for your business scenario, create CDS simple types based on the relevant data elements and release the simple types for internal use (release contract C1). For more information, see [Simple Types](#) in the *ABAP Data Models* documentation.

1.2.3 Exposing a BAPI as OData API Using the ABAP RESTful Application Programming Model

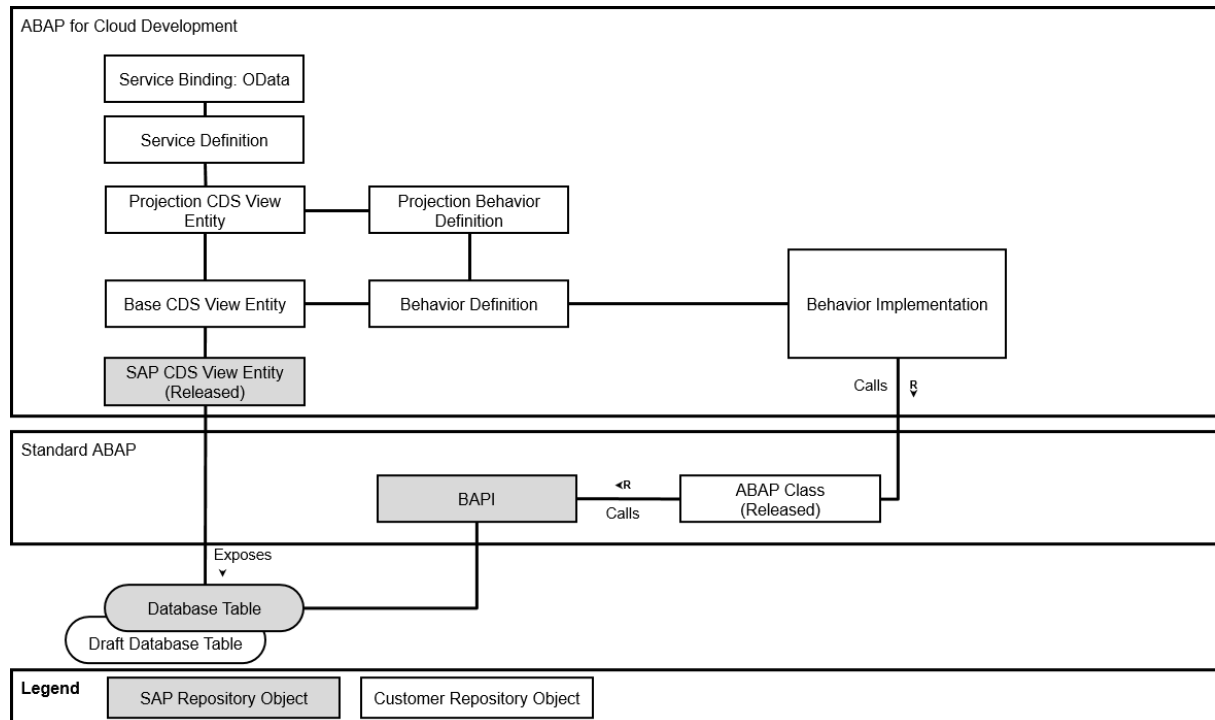
In this documentation, you learn how you can expose a BAPI, which was created in classic ABAP, using a custom wrapper class. The wrapper class encapsulates the BAPI in such a way that it can be exposed as an OData API using the ABAP RESTful Application Programming Model (RAP).

Must-Know: Transactional Apps in RAP

To expose a BAPI as OData API, you create a transactional app for an SAP Fiori UI using RAP. If you aren't familiar with developing transactional apps for an SAP Fiori UI in RAP yet, see [Developing Managed Transactional Apps](#) in the development guide for the ABAP RESTful Application Programming Model (RAP).

BAPI-Based RAP Business Objects

A BAPI-based RAP business object is a RAP business object that consumes business logic of a BAPI by calling it in its behavior implementation. With its CDS data model, the BAPI-based RAP BO retrieves the data from the same persistence as the BAPI.



As a starting point, assume that there's a BAPI that carries out a business task on data stored in a database table whose data is exposed by a released SAP CDS view entity (with release contract C1). If you do not find released CDS view entities for a database table, you can create your own released CDS view entities.

To build a BAPI-based RAP business object, create the following ABAP repository objects:

- CDS view entities on top of existing released CDS view entities, which, in turn, expose the data of the relevant database tables used by the BAPI
- Projections on CDS view entities
- Behavior definition and projection behavior definition to enable CRUD operations for SAP Fiori transactional apps
- A service definition to expose all required CDS view entities and a service binding to expose the service as an OData service

Additionally, to make the BAPI business logic available for consumption in RAP, create an ABAP class and release it with the C1 contract for system-internal use. This class wraps the BAPI written in classic ABAP. The behavior implementation of the business object calls methods from the ABAP wrapper class, which then calls the BAPI.

If you must access artifacts that are not released, you can put them into a behavior definition extension, and this behavior definition extension would then reside in the ABAP language version Standard ABAP. This is a possible approach to consume, for example, non-released authorization objects.

→ Tip

Various generators for building transactional apps in RAP are available (see [OData Web API Service](#)).

1.2.3.1 Creating a BAPI-Based RAP Business Object

For a BAPI-based RAP business object (RAP BO), you model its structure, create the corresponding CDS artifacts, and define the behavior of the RAP BO.

[Reference Scenario: Data Model \[page 39\]](#)

A BAPI-based RAP business object for material classifications is used as a reference scenario. Its data model is based on a composition relationship between the business object entities of material class, class description, and class characteristics.

[Creating CDS View Entities for a BAPI-Based RAP Business Object \[page 41\]](#)

For a BAPI-based RAP business object in the ABAP RESTful Application Programming Model (RAP), you create CDS view entities for the business object entities in your data model.

[Defining the Business Object Structure for a BAPI-Based RAP BO \[page 42\]](#)

When a BAPI-based RAP business object consists of multiple entities, each modeled with a CDS view entity, define the business object structure by creating a composition and adding associations to the CDS source code.

[Defining the Behavior of a BAPI-Based RAP BO \[page 44\]](#)

For BAPI-based business objects (BOs) in the ABAP RESTful Application Programming Model (RAP), you create a behavior definition for your managed RAP BO, including characteristics such as unmanaged save, late numbering, and unmanaged lock master.

1.2.3.1.1 Reference Scenario: Data Model

A BAPI-based RAP business object for material classifications is used as a reference scenario. Its data model is based on a composition relationship between the business object entities of material class, class description, and class characteristics.

Must-Know: Business Objects in the ABAP RESTful Programming Model

The ABAP RESTful Programming Model (RAP) is based on the concept of a business object (BO) to represent real-world artifacts in application development. If you aren't familiar with business objects in RAP yet, see [Business Object](#) in the development guide for the ABAP RESTful Programming Model (RAP).

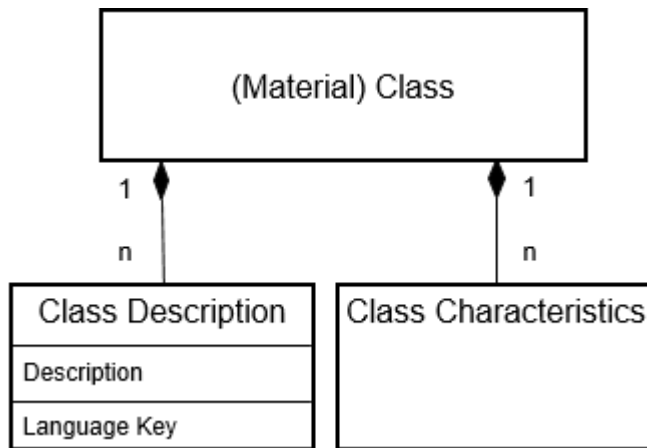
Business Object Entities in the Reference Scenario

In the reference scenario used in this documentation, the following business object entities with selected fields have been chosen for material classifications:

- The class with basic data such as class name, class type (material class), status, and validity dates
- Class description, including a language key for the description
- Class characteristics (such as color)

To avoid using fields that aren't needed for the business use case, you can model a business object in RAP that reduces the API surface to a minimum according to the need-to-know principle.

The following figure shows the composition relationship between the entities of material class, class description, and class characteristics. The material class entity represents the root of the data model, with class description and class characteristics as child entities:



Behavior of the Business Object in the Reference Scenario

In RAP, a behavior definition specifies the behavior of a business object. In this reference scenario, you get an idea of how to define and implement the standard operations **create**, **update**, and **delete** for a material class.

Related Information

[Creating CDS View Entities for a BAPI-Based RAP Business Object \[page 41\]](#)

[Defining the Behavior of a BAPI-Based RAP BO \[page 44\]](#)

1.2.3.1.2 Creating CDS View Entities for a BAPI-Based RAP Business Object

For a BAPI-based RAP business object in the ABAP RESTful Application Programming Model (RAP), you create CDS view entities for the business object entities in your data model.

Must-Know: CDS View Entities in RAP

A CDS view entity is an ABAP repository object that is created using ABAP Core Data Services (ABAP CDS), a technology for defining and consuming SAP HANA-optimized data models. The main purpose of CDS view entities is to select data from one or more data sources, such as database tables. CDS view entities are important entities in RAP and constitute the starting point to model your RAP business object.

If you aren't familiar with ABAP CDS and CDS view entities yet, see [ABAP Data Models](#) and [View Entities](#) in the *ABAP Data Models* documentation. For more information about the role of CDS view entities in RAP, see [Providing Business Object Structure](#) in the development guide for the ABAP RESTful Application Programming Model (RAP).

Creating CDS View Entities for the BAPI-Based RAP BO

The data that is accessed and changed using a BAPI is persisted in database tables. To be able to integrate the BAPI into the data model for RAP, you start with creating CDS view entities that expose the data of these database tables.

To create the data definitions of the CDS view entities, follow the instructions under [Creating a Data Definition](#) in the documentation for ABAP Development Tools.

Example

For the reference scenario used in this documentation, released CDS view entities already exist. Custom CDS view entities have been created on top of the existing released CDS view entities for the classification scenario (material class, class description, class characteristics).

The following entities are available as part of the data model in the reference scenario:

Business Object Entity	Description	Database Table	C1-Released CDS View Entity	Custom CDS View Entity
Class	A class entity with general class data such as the class name, class type (for example, material), status, and validity dates	KLAH	I_CLFNCLASSDEX	ZDSAG_R_CLASSIFICATION
Class description	The class description and the language key	SWOR	I_CLFNCLASSDESCRIPTION	ZDSAG_R_CLASSDESCRIPTION
Class characteristics	Characteristics for classifying objects in a class (such as color)	KSML	I_CLFNCLASSCHARACTERISTICDEX	ZDSAG_R_CLASSCHARACTERS

The class entity represents the root view entity of the data model and the parent entity in a composition. The additional entities for class description and class characteristics are child entities of the class entity.

1.2.3.1.3 Defining the Business Object Structure for a BAPI-Based RAP BO

When a BAPI-based RAP business object consists of multiple entities, each modeled with a CDS view entity, define the business object structure by creating a composition and adding associations to the CDS source code.

Must-Know: Data Model and Business Object Structure in the ABAP RESTful Application Programming Model (RAP)

From a structural perspective, a business object consists of a tree of entities that are linked by compositions. Every entity in this composition tree is an element that is modeled with a CDS view entity. One of the CDS view entities defines the root entity of the managed business object, and it represents the root of a compositional hierarchy.

If you aren't familiar with associations and compositions yet, see [Associations](#) and [Compositions](#) in the *ABAP Data Models* documentation. For more information about defining the data model and business object structure in RAP, see [Defining Data Model and Business Object Structure](#) in the development guide for the ABAP RESTful Application Programming Model (RAP).

Example

This documentation uses a reference scenario with a BAPI-based RAP business object for material classifications. The compositional hierarchy is defined as follows in the root entity of the RAP BO:

Sample Code

```
define root view entity ZDSAG_R_CLASSIFICATION
  as select from I_ClfnClassDEX
  composition [0..*] of ZDSAG_R_CLASSDESCRIPTION as _ClassificationDescription
  composition [0..*] of ZDSAG_R_CLASSCHARACTERS as
    _ClassificationCharacteristic
```

ZDSAG_R_CLASSIFICATION is the root view entity and it defines to-child associations to the description and characteristics entities.

In the child entities, the to-parent association is defined for the composition, as shown in the following example of the description child entity ZDSAG_R_CLASSDESCRIPTION:

Sample Code

```
define view entity ZDSAG_R_CLASSDESCRIPTION
  as select from I_ClfnClassDescription
  association to parent ZDSAG_R_CLASSIFICATION as _Classfications on
    $projection.ClassInternalID = _Classfications.ClassInternalID
```

To be able to access data from additional entities, you can also add more associations. In the following example, the associations refer to the released CDS view entities that are part of the reference scenario. The associations are used to complete the description of the entity model and to get texts such as the class description:

Sample Code

```
define root view entity ZDSAG_R_CLASSIFICATION
" ...
  association [0..*] to I_ClfnClassTypeText as _ClassType
    on _ClassType.ClassType = $projection.ClassType
  association [0..*] to I_ClfnClassDescription as _ClassDescription
    on _ClassDescription.ClassInternalID = $projection.ClassInternalID
  association [0..1] to I_ClfnClassStatusText as _ClassStatus on
    $projection.ClassStatus = _ClassStatus.ClassStatus
    and $projection.ClassType = _ClassStatus.ClassType
    and _ClassStatus.Language = $session.system_language
  association [0..*] to ZDSAG_PRODUCT_HELPER as _ProductHelper on
    $projection.ClassInternalID = _ProductHelper.ClassInternalID
    and $projection.ClassType = _ProductHelper.ClassType
```

1.2.3.1.4 Defining the Behavior of a BAPI-Based RAP BO

For BAPI-based business objects (BOs) in the ABAP RESTful Application Programming Model (RAP), you create a behavior definition for your managed RAP BO, including characteristics such as unmanaged save, late numbering, and unmanaged lock master.

Must-Know: Behavior Definitions in RAP

If you aren't familiar with behavior definitions in RAP yet, see [Defining Standard Behavior](#) in the development guide for the ABAP RESTful Application Programming Model (RAP). For more information about lock definitions in RAP, see [Lock Definition](#).

The RAP framework provides a built-in save sequence, which is the default for any RAP BO with managed implementation type. In some use cases, you might want to replace the default save sequence with custom logic by defining an unmanaged save. For more information, see [Unmanaged Save](#).

Behavior Definitions of BAPI-Based RAP BOs

For the BAPI integration into RAP BOs, you can use RAP BOs with managed implementation type. This means you benefit from the out-of-the-box support for transactional processing that the RAP framework provides. However, because you want to integrate BAPI business logic, you must implement some behavior yourself: save and locking, which you therefore define as unmanaged save and unmanaged lock master.

You can use RAP determinations and validations to include custom logic before saving data records. However, avoid coding any validation that the BAPI logic already covers.

For BAPI-based RAP BOs, you must choose late numbering to set values for primary key fields of entity instances during runtime. With BAPIs, the final key of a new instance is only available after the actual posting of the BAPI. Only late numbering in RAP supports this scenario.

For more information, see [BAPI Consumption in RAP: Transactional Consistency \[page 24\]](#) and [BAPI Consumption in RAP: Late Numbering and Save Sequence \[page 26\]](#).

Example

Unmanaged Save, Late Numbering, and Unmanaged Lock

In the reference scenario used in this documentation, the source code of the behavior definition `ZDSAG_R_CLASSIFICATION` consists of a header information and three definitions for entity behavior. One definition is created for the root entity for a material class. Two more definitions are created for the child entities for class description and class characteristics.

In the header of the behavior definition, define a managed business object with unmanaged save, as in this example:

Sample Code

```
managed with unmanaged save implementation in class zbp_dsag_r_classification
unique;
strict ( 2 );
with draft;
```

In the behavior definition of the individual entities (Class, Description, and Characteristics), you define an unmanaged save as well as late numbering. The following example shows this for the **Classification** root entity. In this root entity, also define the unmanaged lock master:

Sample Code

```
define behavior for ZDSAG_R_CLASSIFICATION alias Classification
with unmanaged save
late numbering
draft table zdsag_class_d
etag master ClassLastChangedDateTime
lock master unmanaged
total etag ClassLastChangedDateTime
authorization master ( global, instance )
```

Validations and Determinations

In the root entity, you can also define validations and determinations, as in the following example:

Sample Code

```
define behavior for ZDSAG_R_CLASSIFICATION alias Classification
" ...
validation check_if_classtype_exists on save { create; field ClassType; }
validation simulate_create on save { create; update; }
validation simulate_delete on save { delete; }
determination setDefaultValues on modify { create; }
```

The validation `check_if_classtype_exists` checks whether a class type with the same name already exists. The validations `simulate_create` and `simulate_delete` trigger the test run of the BAPI. The test run allows error messages and warnings to be returned to the user interface if there's an issue, for example, with user input. With the determination `setDefaultValues`, you can define default values for fields when a new class is created. For example, you can set the logon language as the default language for the description. You must implement validations and determinations later as part of the behavior implementation.

1.2.3.2 Creating and Releasing a Wrapper Class for a BAPI

To consume BAPI business logic in the ABAP RESTful Application Programming Model (RAP), you can create a custom ABAP class and release it for internal use (release contract C1). This ABAP class wraps the BAPI written in standard ABAP and makes it available for internal use in ABAP for Cloud development.

[Creating a Wrapper Class for Calling a BAPI \[page 46\]](#)

Using the example class `zdsag_bapi_class_create`, you learn how to create a wrapper class that encapsulates functionality to call BAPIs. The wrapper class also helps map CDS view entity properties to BAPI structures and supports conversions.

[Releasing the Wrapper Class for Internal Use \(Release Contract C1\) \[page 49\]](#)

After creating a wrapper class for a BAPI, release the class for internal use, so that it can be used for ABAP for Cloud development.

1.2.3.2.1 Creating a Wrapper Class for Calling a BAPI

Using the example class `zdsag_bapi_class_create`, you learn how to create a wrapper class that encapsulates functionality to call BAPIs. The wrapper class also helps map CDS view entity properties to BAPI structures and supports conversions.

→ Recommendation

While a wrapper class isn't mandatory from a technical point of view, we recommend that you create one for your BAPI for internal use (release contract C1). You can use the class for integrating BAPI business logic into your business objects in the ABAP RESTful Application Programming Model (RAP).

Wrapper Class in Short

An ABAP class that serves as a wrapper for a BAPI comprises the following:

- A private method for calling the BAPI
This method calls the BAPI and limits the number of parameters to those needed for your use case. Many BAPIs provide a large number of parameters that might not be required in a particular use case. Many BAPIs provide a parameter `testrun`, which you need to run validations before save for your RAP business object. You can also use the BAPI parameter `return` to return BAPI messages, which you can pass on to the UI layer.
For more information, see [BAPI Consumption in RAP: Transactional Consistency \[page 24\]](#) and [BAPI Consumption in RAP: Messages \[page 29\]](#).
- A public method for mapping BAPI structures to CDS entity properties
Such a mapping also includes the conversions between the internal and external BAPI data formats (see [BAPI Consumption in RAP: Conversions \[page 32\]](#)).
Some BAPIs don't directly return the ID of the instance that they create. You can mitigate this situation by retrieving the ID from the buffer (see the following example).

Example

In class `zdsag_bapi_class_create`, you can find the following methods:

- `bapi_class_create` (in the private section) for calling the BAPI `BAPI_CLASS_CREATE`.
- `map_4_bapi_class_create` (in the public section) for mappings and for retrieving a class ID from the buffer

Method `bapi_class_create` for Calling the BAPI

In the `bapi_class_create` method, you call the BAPI `BAPI_CLASS_CREATE`. In this example, only a small selection of BAPI parameters is used for the sake of simplicity. The method uses the `testrun` and `return` parameters for validations and messages:

Sample Code

```
METHOD bapi_class_create.  
  CALL FUNCTION 'BAPI_CLASS_CREATE'  
    EXPORTING  
      classbasicdata      = classbasicdata  
      classnumnew         = classnumnew  
      classtypenew        = classtypenew  
      testrun             = testrun  
    TABLES  
      classcharacteristics = classcharacteristics  
      classdescriptions    = classdescriptions  
      return               = return  
    EXCEPTIONS  
      communication_failure = 1  
      system_failure       = 2  
      OTHERS               = 3.  
  IF testrun = abap_true.  
    " Required to clean the buffer manually  
    CALL FUNCTION 'BUFFER_REFRESH_ALL'.  
  ENDIF.  
ENDMETHOD.
```

Method `map_4_bapi_class_create` for Mappings and Conversions

In the method `map_4_bapi_class_create`, you map the CDS entity properties to BAPI structures. In the following example, the basic data of the material class are mapped:

Sample Code

```
" Basic Data from Root entity 'ZDSAG_R_CLASSIFICATION'-----  
ls_basic_data-valid_from = iv_classification-ValidityStartDate.  
ls_basic_data-valid_to   = iv_classification-ValidityEndDate.  
ls_basic_data-status     = iv_classification-ClassStatus.
```

BAPIs aren't designed for UI consumption, so you as a developer must also take care of conversions. For example, for the material descriptions from the entity `ZDSAG_R_CLASSDESCRIPTION`, the language codes must be converted from ABAP format to ISO format:

Sample Code

```
" Description from entity 'ZDSAG_R_CLASSDESCRIPTION'-----  
LOOP AT it_descriptions ASSIGNING FIELD-SYMBOL(<fs_description>).  
  ls_description-langu = <fs_description>-LanguageEdit.  
  CALL FUNCTION 'LANGUAGE_CODE_SAP_TO_ISO'  
    EXPORTING  
      sap_code = ls_description-langu  
    IMPORTING  
      iso_code = ls_description-langu_iso.
```

```

ls_description-catchword = <fs_description>-ClassDescription.
APPEND ls_description TO lt_descriptions.
CLEAR ls_description.
ENDLOOP.

```

After mappings and the conversions, the `map_4_bapi_class_create` method calls the `bapi_class_create` method (which, in turn, calls the BAPI):

Sample Code

```

bapi_class_create( EXPORTING classtypenew      = iv_classification-
ClassType
                  classnumnew      = iv_classification-
Class
                  classbasicdata    = ls_basic_data
                  testrun           = test_run
                  CHANGING classdescriptions = lt_descriptions
                  classcharacteristics = lt_characteristics
                  return            = return ).

```

You also use the method `map_4_bapi_class_create` for special handling, such as the retrieval of the classification ID: In this example, the BAPI doesn't directly return the ID of the classification instance that was created as parameter. You can address this situation by retrieving the ID from the buffer using the function module `CLSE_SELECT_KLAH_VIA_NAMERANGE` and returning the ID with method `map_4_bapi_class_create`:

Sample Code

```

IF test_run = abap_false. "Retrieve ID only during save, not during a
simulation (test run)
" BAPI does not explicitly return the "Classification ID -CLINT", so access
that information from buffer
ls_range-sign   = 'I'.
ls_range-option = 'EQ'.
ls_range-low    = iv_classification-Class.
APPEND ls_range TO lt_range.

CALL FUNCTION 'CLSE_SELECT_KLAH_VIA_NAMERANGE'
TABLES
  imp_exp_klah   = lt_klah
  namerange      = lt_range
EXCEPTIONS
  no_entry_found = 1
  OTHERS        = 2.
IF sy-subrc = 0.
  DATA(ls_class) = VALUE #( lt_klah[ klagt = iv_classification-ClassType
                                     class = iv_classification-Class ]
OPTIONAL ).
  IF ls_class IS NOT INITIAL.
    ev_classinternalid = ls_class-clint.
  ENDIF.
ELSE.
  ASSERT 1 = 2.
ENDIF.
ENDIF.

```

The retrieved ID in `ev_classinternalid` is later mapped to the classification BO instance key.

1.2.3.2 Releasing the Wrapper Class for Internal Use (Release Contract C1)

After creating a wrapper class for a BAPI, release the class for internal use, so that it can be used for ABAP for Cloud development.

You can release an ABAP repository object by setting the API state in its properties in ABAP Development Tools. The C1 contract is for development objects that are intended for system-internal consumption. The assignment of C1 is required for any development object used in an extension implementation, so this API state is the right choice for custom wrapper classes for BAPIs.

Related Information

[Contract C1: Use System-Internally
Releasing Development Objects](#)

1.2.3.3 Developing the Business Logic for a BAPI-Based RAP BO

For a business object (BO) in the ABAP RESTful Application Programming Model (RAP), you can integrate BAPI business logic into the behavior implementation of the BO.

[Implementing Validations for BAPI-Based RAP BOs \[page 50\]](#)

For a business object in the ABAP RESTful Application Programming Model (RAP BO), you can reuse BAPI business logic for validations by calling the BAPI in simulation mode (test run).

[Implementing BAPI Call, Late Numbering, and Unmanaged Save for BAPI-Based RAP BOs \[page 51\]](#)

When you implement the behavior of a BAPI-based business object in the ABAP RESTful Application Programming Model (RAP), you must implement late numbering and the BAPI call for the `create` operation.

[Implementing an Unmanaged Lock for BAPI-Based RAP Business Objects \[page 52\]](#)

For BAPI-based business objects in the ABAP RESTful Application Programming Model (RAP BO), you implement unmanaged locks and call the BAPI business logic for locking in the behavior implementation of the RAP BOs.

[Adding Update and Delete Operations to BAPI-Based RAP Business Objects \[page 53\]](#)

To add `update` and `delete` operations to BAPI-based business objects in the ABAP RESTful Application Programming Model (RAP BOs), you need wrapper classes for the relevant update and delete BAPIs. In addition, you must implement an unmanaged locking. `Update` operations require additional coding for identifying the data that has been changed.

1.2.3.3.1 Implementing Validations for BAPI-Based RAP BOs

For a business object in the ABAP RESTful Application Programming Model (RAP BO), you can reuse BAPI business logic for validations by calling the BAPI in simulation mode (test run).

Must-Know: Validations in RAP

If you aren't familiar with validations in RAP yet, see [Validations](#) in the development guide for the ABAP RESTful Application Programming Model (RAP).

Validations for BAPI-Based RAP BOs

For BAPI-based RAP BOs, you can implement new validations in the behavior pool of the RAP BO. For example, you can create a validation that checks whether an instance already exists. However, the BAPIs that you want to integrate into RAP BOs often include their own validations. To avoid duplicating business logic, you can call the relevant BAPI in simulation mode from the behavior implementation of the RAP BO. This approach ensures that the BAPI validations run and that the system returns BAPI error messages and warnings to the user interface.

For more information, see [BAPI Consumption in RAP: Messages \[page 29\]](#).

Example

In the reference scenario used in this documentation, the method `check_if_classtype_exists` is an example of a quick validation in RAP before the BAPI is called. The validation checks whether the classification type that was entered exists (for example, a material class). If an invalid classification type is entered, the system returns an error message to the user interface. This validation is a simple check that you can easily implement as an application expert.

With the `simulate_create` method in the reference scenario, the validations that are part of the BAPI code are reused. This approach eliminates the need to manually implement these validations in RAP. The `simulate_create` method calls the BAPI in test run. In the method `simulate_create`, the method `map_4_bapi_class_create` of the BAPI wrapper class `zdsag_bapi_class_create` is called with parameter `test_run = abap_true`:

Sample Code

```
METHOD simulate_create.  
" ...  
zdsag_bapi_class_create=>map_4_bapi_class_create(  
    EXPORTING iv_classification = CORRESPONDING  
    #( <fs_classification> )  
        it_descriptions      = mapped_descriptions  
        it_characteristics   = mapped_characteristics
```

```

        test_run          = abap_true
IMPORTING ev_classinternalid = DATA(rs_classinternalid)
CHANGING  return            = lt_return ).

```

1.2.3.3.2 Implementing BAPI Call, Late Numbering, and Unmanaged Save for BAPI-Based RAP BOs

When you implement the behavior of a BAPI-based business object in the ABAP RESTful Application Programming Model (RAP), you must implement late numbering and the BAPI call for the create operation.

Numbering and Save Sequence for BAPI-Based RAP BOs

With BAPIs, the final key of a newly created BO instance is only available after the actual BAPI posting has run. Therefore, you must use late numbering in RAP to assign the ID to the new BAPI-managed RAP BO instance.

The BAPI is called after the point of no return in the save sequence, which includes the following methods:

1. **ADJUST_NUMBERS**: For the create operation, you must call the BAPI here. After this point, you can no longer handle errors or return keys.
2. **SAVE_MODIFIED**: For the create operation, this save method isn't required because the BAPI has already written the data to the database. However, you still need the **SAVE_MODIFIED** method for change and delete operations.

For more information, see [BAPI Consumption in RAP: Late Numbering and Save Sequence \[page 26\]](#).

Example

The reference scenario for this documentation uses BAPIs for material classification, in particular, the BAPI **BAPI_CLASS_CREATE** for creating material classes.

In the reference scenario, when the **ADJUST_NUMBERS** method runs during the save sequence, the UI input is read and mapped to the corresponding BAPI structures. The BAPI **BAPI_CLASS_CREATE** is called through its wrapper class **zdsag_bapi_class_create**. For the actual save of the data, it's called **without** test run, that is, **test_run = abap_false**:

Sample Code

```

METHOD adjust_numbers.
" ...
LOOP AT classifications ASSIGNING FIELD-SYMBOL(<fs_classification>) GROUP BY
<fs_classification>-%tky.
"Reading and mapping material descriptions and characteristics
    LOOP AT links_descriptions ASSIGNING FIELD-SYMBOL(<link_description>)
    USING KEY id WHERE source-%tky = <fs_classification>-%tky.
        DATA(description) = descriptions[ KEY id
                                         %tky = <link_description>-target-
                                         %tky ].

```

```

        APPEND CORRESPONDING #( description ) TO mapped_descriptions
        ASSIGNING FIELD-SYMBOL(<mapper_description>).
    ENDLOOP.
    LOOP AT links_characteristics ASSIGNING FIELD-
    SYMBOL(<links_characteristic>) USING KEY id WHERE source-%tky =
    <fs_classification>-%tky.
        DATA(characteristic) = Characteristics[ KEY id
                                                %tky = <links_characteristic>-
        target-%tky ].
        APPEND CORRESPONDING #( characteristic ) TO mapped_characteristics.
    ENDLOOP.
    "BAPI call using wrapper class zdsag_bapi_class_create
    TRY.
        zdsag_bapi_class_create=>map_4_bapi_class_create(
            EXPORTING iv_classification = CORRESPONDING
            #( <fs_classification> )
                it_descriptions      = mapped_descriptions
                it_characteristics   = mapped_characteristics
                test_run             = abap_false
            IMPORTING ev_classinternalid = DATA(rs_classinternalid)
            CHANGING  return          = lt_return ).
        CATCH cx_root.
            " handle exception
        ENDTRY.
    "BAPI messages will be shown on UI -----
    handle_bapi_return( EXPORTING header_tky = VALUE #( %is_draft =
    <fs_classification>-%is_draft
    <fs_classification>-%pid
    rs_classinternalid )
    return          = lt_return
    IMPORTING reported = reported ).

    "Returned key of the new material class instance is mapped to the ID of the
    new RAP BO instance
    mapped_classification[ %pid = <fs_classification>-%pid ]-
    ClassInternalID = rs_classinternalid.
    ENDLOOP.
ENDMETHOD.

```

1.2.3.3.3 Implementing an Unmanaged Lock for BAPI-Based RAP Business Objects

For BAPI-based business objects in the ABAP RESTful Application Programming Model (RAP BO), you implement unmanaged locks and call the BAPI business logic for locking in the behavior implementation of the RAP BOs.

Locking for BAPI-Based RAP BOs

Since BAPIs come with their own business logic and locking mechanisms, you reuse the relevant function modules for locking. To do so, include the function modules in a wrapper class, so that you can call them through the wrapper class. Because the locking behavior for the BAPI-based RAP BO is non-standard, use an unmanaged lock handler in your custom RAP BO implementation.

For more information, see [BAPI Consumption in RAP: Locking \[page 36\]](#).

Example

The BAPI for material classification in the reference scenario uses the function module `ENQUEUE_ECKSSKXE` for locking. For this function module, a wrapper class, `zdsag_classification_utility`, has been created and released for internal use (release contract C1).

Locking is defined as unmanaged in the behavior definition of the RAP BO root entity (see also [Defining the Behavior of a BAPI-Based RAP BO \[page 44\]](#)):

Sample Code

```
define behavior for ZDSAG_R_CLASSIFICATION alias Classification
" ...
lock master unmanaged
```

The `lock` method is defined and implemented as part of the behavior pool of the BAPI-based RAP BO. In the behavior pool, the `lock` method calls the function module `ENQUEUE_ECKSSKXE` through its wrapper class `zdsag_classification_utility`. An error is returned if the material class instance is already locked:

Sample Code

```
METHOD lock.
" ...
zdsag_classification_utility=>enqueue_classification( EXPORTING classtype =
<fs_classification>-ClassType                                class      =
<fs_classification>-class                                     IMPORTING lock_error =
DATA(lock_error) ).
```

1.2.3.3.4 Adding Update and Delete Operations to BAPI-Based RAP Business Objects

To add update and delete operations to BAPI-based business objects in the ABAP RESTful Application Programming Model (RAP BOs), you need wrapper classes for the relevant update and delete BAPIs. In addition, you must implement an unmanaged locking. Update operations require additional coding for identifying the data that has been changed.

Wrapper Classes for the BAPIs

To add update and delete operations to a BAPI-based RAP BO, you proceed in a similar way as for the create operation:

- Create an ABAP class for wrapping the relevant BAPIs (see also [Creating a Wrapper Class for Calling a BAPI \[page 46\]](#)).
- Release the wrapper class for internal use (see also [Releasing the Wrapper Class for Internal Use \(Release Contract C1\) \[page 49\]](#)).
- Call the BAPIs through their wrapper classes from the behavior pool of the RAP BO.

Locking

Before users can update or delete data, the relevant entities must be locked. In a BAPI-based RAP BO, you can use the original SAP lock object of the BAPI (see [BAPI Consumption in RAP: Locking \[page 36\]](#)). For the update and delete operations, you implement an unmanaged locking in RAP and reuse the relevant enqueue function modules of the BAPIs. For more information, see [Implementing an Unmanaged Lock for BAPI-Based RAP Business Objects \[page 52\]](#).

Reading Values of Changed Fields

Implementing the update operation can be challenging. In addition to entity instance data, you also need information about the changed fields. Most of the BAPIs require to provide an x-structure to identify field changes (also allowing to set fields to their initial / blank state). Before SAP S/4HANA 2025, you cannot access aggregated change flags for data records modified in a given transaction. A workaround can be to perform a privileged select from the CDS view entity and to calculate the delta between the input data and the persistent data.

As of SAP S/4HANA 2025, you can use the EML statement `READ ENTITY` with the addition `WITH CHANGES` to read the values of changed fields. In addition to entity instance data, information about the operation and changed fields is also returned. To hold that additional information, the result type `STRUCTURE/TABLE FOR READ CHANGES` is available. For more information about the `READ ENTITIES` statement and its `WITH CHANGES` addition, see [READ ENTITY, ENTITIES, WITH CHANGES \(ABAP Keyword Documentation\)](#).

Note

Note, however, that not all BAPIs can benefit from this new syntax, including the reference example used in this documentation.

Example

In the reference scenario used in this documentation, the wrapper classes `zdsag_bapi_class_change` and `zdsag_bapi_class_delete` have been created and released for the BAPIs `BAPI_CLASS_CHANGE` and `BAPI_CLASS_DELETE`.

Similar to the create operation, in the class `zdsag_bapi_class_change`, the method `map_4_bapi_class_change` reads and processes input data and performs the necessary mapping between CDS view entity and BAPI structures. Similar to the create operation, validations for the update and delete

operations are used that integrate the simulation mode (test run) of the respective BAPIs. Again, similar to the `create` operation, any errors returned from the BAPIs are passed on to the UI.

For the `update` operation, the method `zdsag_get_class_detail=>get_classification_details` is used to read the existing persisted data. The BAPI `BAPI_CLASS_CHANGE` is called by the method to perform the actual change operation.

1.2.3.4 Creating a Projection Layer and a Business Service for a BAPI-Based RAP Business Object

Complete a BAPI-based business object in the ABAP RESTful Application Programming Model (RAP BOs): Add a projection layer and define a business service based on these projections.

Must-Know: Projection Layer and Business Services in RAP

If you aren't familiar yet with projections, business services and service bindings in RAP, see [Business Object Projection](#), [Service Definition](#), and [Service Binding](#) in the development guide for the ABAP RESTful Application Programming Model (RAP).

Projection Layer, Service Definition and Binding for a BAPI-Based RAP BO

For a BAPI-based RAP BO, follow the same steps as for any other RAP BO: Define the relevant projections, the business service, and service binding. Choose binding type `ODATA V4 - UI`. As a result, you get a basic transactional SAP Fiori app based on this RAP BO.

1.2.4 Adding BAPI Business Logic to a RAP Business Object Using RAP Actions

Using the concept of RAP actions, you can add BAPI logic to a business object (BO) in the ABAP RESTful Application Programming Model (RAP).

Adding a RAP action is one possibility to consume BAPIs in RAP. You can also integrate BAPI logic tighter into RAP BOs by building a BAPI-based RAP BO. A BAPI-based RAP BO consumes business logic from a BAPI by calling it in its behavior implementation and that retrieves the data from the same persistency as the BAPI (see [Creating a BAPI-Based RAP Business Object \[page 39\]](#)).

Example

In this example, the aim is to add business logic to a RAP BO so that it can assign a material class to a material. In classic ABAP, such an assignment is performed using transactions [Create Material](#) or [Change Material](#) (transaction codes MM01 and MM02). The underlying available BAPI `BAPI_OBJCL_CREATE` with its material class assignment can be integrated into a RAP BO by adding an action to the behavior definition of the RAP BO.

Note

In this example, the RAP action is added to a **BAPI-based** RAP business object, which already comprises BAPI business logic as part of its behavior implementation. This approach was taken to get an end-to-end reference scenario. Using a BAPI-based RAP business object, though, isn't a prerequisite for adding BAPI business logic as a RAP action. You can also use a "standard" RAP business object and add BAPI business logic as a RAP action.

Related Information

[Reference Scenario \[page 16\]](#)

1.2.4.1 Example: Wrapper Class for the BAPI `BAPI_OBJCL_CREATE`

Before you can add BAPI business logic to a RAP business object using RAP actions, you must wrap the BAPI in an ABAP class and release the class for internal use.

The BAPI `BAPI_OBJCL_CREATE` provides business logic to assign a material class to a material. The BAPI is written in standard ABAP. To consume the BAPI in the ABAP RESTful Application Programming Model (RAP), you must build a wrapper class for internal release (release contract C1).

In the reference scenario used for this documentation, a wrapper class for the BAPI was created and released (class `zdsag_bapi_objcl_create`). With method `bapi_objcl_create` of this class, you call the BAPI:

Sample Code

```
METHOD bapi_objcl_create.  
  CALL FUNCTION 'BAPI_OBJCL_CREATE'  
    EXPORTING  
      classnumnew      = classnumnew  
      classtypenew     = classtypenew  
      keydate          = keydate  
      objectkeynew      = objectkeynew  
      objectkeynew_long = objectkeynew_long  
      objecttablenew    = objecttablenew  
    TABLES  
      return           = return  
    EXCEPTIONS  
      communication_failure = 1  
      system_failure       = 2
```

```

        OTHERS                = 3.
    IF sy-subrc <> 0.
    ENDIF.
ENDMETHOD.

```

With the method `map_4_bapi_objcl_create`, you call the function module for material number conversion and you call the method `bapi_objcl_create`:

Sample Code

```

METHOD map_4_bapi_objcl_create.
    DATA: lv_object          TYPE objnum.
    CALL FUNCTION 'CONVERSION_EXIT_MATN1_INPUT'
        EXPORTING
            input              = objectkeynew
        IMPORTING
            output             = lv_object
    EXCEPTIONS
        length_error = 1
        OTHERS       = 2.
    bapi_objcl_create( EXPORTING
                        classnumnew = classnumnew
                        classtypenew = classtypenew
                        keydate = sy-datum
                        objectkeynew = lv_object
                        objectkeynew_long = CONV #( lv_object )
                        objecttablenew = 'MARA'
                        CHANGING
                            return = return
                        ).
ENDMETHOD.

```

Related Information

[Creating a Wrapper Class for Calling a BAPI \[page 46\]](#)

[Releasing the Wrapper Class for Internal Use \(Release Contract C1\) \[page 49\]](#)

[Reference Scenario \[page 16\]](#)

1.2.4.2 Adding a RAP Action for the BAPI Logic to the Behavior Definition

As part of the integration of a BAPI into a business object in the ABAP RESTful Application Programming Model (RAP), you define a RAP action in the behavior definition of the RAP BO that calls the BAPI.

Must-Know: Actions in RAP

An action in RAP is a non-standard modifying operation that is part of the business logic. For more information, see [Actions](#) in the development guide for the ABAP RESTful Application Programming Model (RAP).

Example

In the reference scenario used in this documentation, the following code was added to the behavior definition ZDSAG_R_CLASSIFICATION of the RAP BO:

Sample Code

```
define behavior for ZDSAG_R_CLASSIFICATION alias Classification
" ...
action ( features : instance ) assign_class_to_material parameter
ZDSAG_PARAMETER_MATNR_DETAIL result [1] $self;
```

The action triggers the `assign_class_to_material` method, which is implemented as part of the behavior implementation of the classification RAP BO (see [Implementing a RAP Action with Integrated BAPI Business Logic \[page 58\]](#)).

1.2.4.3 Implementing a RAP Action with Integrated BAPI Business Logic

To be able to call BAPI business logic in a RAP business object as an action, you must implement a method for the action that prepares the BAPI call. The actual BAPI call is performed in the late save phase.

Overview

For the action, you implement the corresponding method for the action in the behavior pool of the RAP BO. Such an implementation includes reading relevant data, checking authorizations, locking, and preparing the data for a later save. The BAPI itself is then called from the `save_modified` method during the late save phase. If the BAPI call has been successful, the OData runtime ends the save sequence with a commit.

Example

In the reference scenario used in this documentation, an action that assigns a material class to materials was implemented using the business logic of the BAPI `BAPI_OBJCL_CREATE`. With the method `assign_class_to_material` in the behavior implementation `zbp_dsag_r_classification`, the following was implemented:

- **Read data:** The method reads the relevant data (material class, internal class ID, class type) for the provided keys for the `Classification` entity of the RAP BO `zdsag_r_classification`.
- **Authority check:** The method performs an authority check for assigning material class to materials. For the authority checks, the method calls `is_auth_to_create_mat_view` from the `zcl_dsag_class_auth_check` class, a released wrapper class where the relevant authority checks of the BAPIs are called (see also [BAPI Consumption in RAP: Authorizations \[page 33\]](#)).

- **Lock:** The method tries to lock the relevant materials before these materials are used for the class assignment.
For locking, the method calls `enqueue_product` from the released wrapper class `zdsag_product_utility` (see also [BAPI Consumption in RAP: Locking \[page 36\]](#)).
- **Prepare data:** The method prepares the necessary data for the assignment action to be processed in the `save_modified` method during the late save phase. The data preparation includes storing the class internal ID, class type, and material number in the internal table `it_assignment_details` of the `zbp_dsag_r_classification` class. The internal table is used as buffer table, so that the data can be processed later during the late save phase.

Sample Code

```

METHOD assign_class_to_material.
  READ ENTITIES OF zdsag_r_Classification IN LOCAL MODE
    ENTITY Classification
    FIELDS ( Class ClassInternalID ClassType )
    WITH VALUE #( FOR key IN keys
                  ( %tky          = key-%tky
                    ClassInternalID = key-
classinternalid )
                )
    RESULT DATA(lt_classifications).
  LOOP AT lt_classifications ASSIGNING FIELD-SYMBOL(<ls_classification>).
    ASSIGN keys[ KEY entity
                  ClassInternalID = <ls_classification>-ClassInternalID ] TO
FIELD-SYMBOL(<ls_key>).
    " Authority check for creating material view
    DATA(is_allowed)
= zcl_dsag_class_auth_check=>is_auth_to_create_mat_view( <ls_key>-%param-
object ).
    IF is_allowed = abap_false.
      APPEND VALUE #( %tky = <ls_classification>-%tky ) TO failed-
classification.
      APPEND VALUE #( %tky = <ls_classification>-%tky ) TO reported-
classification ASSIGNING FIELD-SYMBOL(<reported>).
      <reported>-%msg = new_message( id
=
zcl_dsag_class_auth_check=>mc_message_id
                                number = 005
                                severity =
if_abap_behv_message=>severity-error ).
      CONTINUE.
    ENDIF.
    " Try to lock Product or Products before taking for assignment of
classification to Product/Products(1:N)
    zdsag_product_utility=>enqueue_product( EXPORTING i_matnr = <ls_key>-%
param-object
                                           IMPORTING lock_error =
DATA(lock_error) ).
    IF lock_error = abap_true.
      APPEND VALUE #( %tky = <ls_classification>-%tky ) TO failed-
classification.
      APPEND VALUE #( %tky = <ls_classification>-%tky ) TO reported-
classification ASSIGNING <reported>.
      <reported>-%msg = new_message( id
=
zbp_dsag_r_classification=>lc_msg_class
                                number = 002
                                severity =
if_abap_behv_message=>severity-error
                                v1      = <ls_key>-%param-object
                                v2      = sy-msgv1 ).
      CONTINUE.
    ENDIF.
    " For later processing of Assignment Action in late SAVE_MODIFIED phase

```

```

        zbp_dsag_r_classification=>ls_assignment_detail-classinternalid =
<ls_classification>-Class.
        zbp_dsag_r_classification=>ls_assignment_detail-classtype      =
<ls_classification>-ClassType.
        zbp_dsag_r_classification=>ls_assignment_detail-material_number =
<ls_key>-%param-object.
        APPEND zbp_dsag_r_classification=>ls_assignment_detail TO
zbp_dsag_r_classification=>it_assignment_details.
    ENDMETHOD.
    " Read changed data for action result
    result = VALUE #( FOR Classification IN lt_Classifications
        ( ClassInternalID      = Classification-
ClassInternalID
        %param-ClassInternalID = Classification-
ClassInternalID ) ).
    ENDMETHOD.

```

In the `save_modified` method of the behavior implementation, the `map_4_bapi_objcl_create` method from the released wrapper class `zdsag_bapi_objcl_create` for the BAPI is called:

Sample Code

```

METHOD save_modified.
" ...
LOOP AT zbp_dsag_r_classification=>it_assignment_details ASSIGNING FIELD-
SYMBOL(<fs_assignment>).
    TRY.
        zdsag_bapi_objcl_create=>map_4_bapi_objcl_create(
            EXPORTING classnumnew = <fs_assignment>-classinternalid
                    classtypenew = <fs_assignment>-ClassType
                    objectkeynew = CONV #( <fs_assignment>-material_number )
            CHANGING return      = lt_return ).
        CATCH cx_root          ##NO_HANDLER.
    ENDTRY.
    handle_bapi_return( EXPORTING header_tky = VALUE #( ClassInternalID =
<fs_assignment>-ClassInternalID )
                        return      = lt_return
                        IMPORTING reported = reported ).
ENDLOOP.

```

This method `map_4_bapi_objcl_create` calls the BAPI `bapi_objcl_create`, which performs the material assignment.

1.2.5 Extensibility for BAPI-Based RAP Business Objects

A BAPI-based RAP business object (BO) can be extended using the extensibility options provided by the ABAP RESTful Application Programming Model (RAP) and ABAP Cloud.

With a BAPI-based RAP BO, the BAPI takes care of saving all the data to database tables during an unmanaged save.

Customer Enhancements for BAPIs in Classic ABAP

A BAPI allows the following customer enhancements in classic ABAP:

- Field extensions: You can add more fields to the data model, that is, the relevant database tables and the related BAPI structures.
- Business logic extensions: In classic ABAP, logical extensibility of BAPIs is implemented using Business Add-Ins (BADIs). BADIs can be called by the BAPI, eliminating the need for customer modifications.

These types of enhancements can also be realized for BAPI-based RAP business objects using the extensibility concepts of the RAP framework, provided some constraints are considered.

Extensions for Business Objects in RAP

RAP extensions enable you to extend existing RAP BOs to adapt their functionality to your specific business requirements. Depending on your business use case, you can extend the following with a RAP business object:

- CDS data model extensions: You can add more fields to the data model.
- Behavior extensions: You can add new validations or determinations, actions, and functions.
- Node extensions: You can extend the BO with additional BO entities and extend the service definition to include the new BO nodes.

For more information about RAP extensions, see [RAP Extensions](#).

Extensions for BAPI-Based RAP BOs

Logical or Behavior Extensibility

Logical extensibility for BAPI-based RAP BOs can be achieved using BADIs for the BAPI, if available. However, it's important to note that the BAPI must call the customer exits during the BAPI test run to ensure that validations implemented through the BADI are executed during the early save phase of the BAPI-based RAP BO.

You can also extend the BAPI-based RAP BO using the RAP behavior extensions such as additional determinations and validations. They can be added to the behavior definition of the RAP BO and be implemented in the behavior pool of the RAP BO.

Node Extensibility

In classic ABAP, the concept of node extensibility for BAPIs doesn't exist. In a BAPI-based RAP BO, however, you can also add your own extension nodes (entities).

Field Extensibility

For field extensibility, you combine the custom enhancements for BAPIs with the field extensibility in RAP: For the BAPI, you use the BAPI table extensions for adding fields in standard ABAP. For RAP, you must add extension fields to the database tables and CDS view entities. Building extensions in RAP follows a bottom-up approach, meaning you first define your extensions on the base BO layer, and then continue to the projection

layer and the service exposure level. Extension fields are added as extension to extension include structures, which are embedded in the active and draft database tables, so that the extension must be defined only once for a draft-enabled RAP BO.

As a prerequisite, you must first determine whether the database table and the relevant CDS view entities are enabled for extensibility, including any released CDS view entities that you use. Finally, you must define a mapping from the RAP extension include structure to the BAPI table extensions.

1.2.6 Using the ABAP Test Cockpit as a Supporting Tool for Adopting ABAP Cloud

Use the ABAP Test Cockpit (ATC) to establish governance for Cloud-ready development by enforcing the syntax of ABAP for Cloud development.

More Information

[How the ABAP Test Cockpit supports you to adopt ABAP Cloud](#) 

[Checking Quality of ABAP Code with ATC](#)

Important Disclaimers and Legal Information

Hyperlinks

Some links are classified by an icon and/or a mouseover text. These links provide additional information.
About the icons:

- Links with the icon  : You are entering a Web site that is not hosted by SAP. By using such links, you agree (unless expressly stated otherwise in your agreements with SAP) to this:
 - The content of the linked-to site is not SAP documentation. You may not infer any product claims against SAP based on this information.
 - SAP does not agree or disagree with the content on the linked-to site, nor does SAP warrant the availability and correctness. SAP shall not be liable for any damages caused by the use of such content unless damages have been caused by SAP's gross negligence or willful misconduct.
- Links with the icon  : You are leaving the documentation for that particular SAP product or service and are entering an SAP-hosted Web site. By using such links, you agree that (unless expressly stated otherwise in your agreements with SAP) you may not infer any product claims against SAP based on this information.

Videos Hosted on External Platforms

Some videos may point to third-party video hosting platforms. SAP cannot guarantee the future availability of videos stored on these platforms. Furthermore, any advertisements or other content hosted on these platforms (for example, suggested videos or by navigating to other videos hosted on the same site), are not within the control or responsibility of SAP.

Beta and Other Experimental Features

Experimental features are not part of the officially delivered scope that SAP guarantees for future releases. This means that experimental features may be changed by SAP at any time for any reason without notice. Experimental features are not for productive use. You may not demonstrate, test, examine, evaluate or otherwise use the experimental features in a live operating environment or with data that has not been sufficiently backed up. The purpose of experimental features is to get feedback early on, allowing customers and partners to influence the future product accordingly. By providing your feedback (e.g. in the SAP Community), you accept that intellectual property rights of the contributions or derivative works shall remain the exclusive property of SAP.

Example Code

Any software coding and/or code snippets are examples. They are not for productive use. The example code is only intended to better explain and visualize the syntax and phrasing rules. SAP does not warrant the correctness and completeness of the example code. SAP shall not be liable for errors or damages caused by the use of example code unless damages have been caused by SAP's gross negligence or willful misconduct.

Bias-Free Language

SAP supports a culture of diversity and inclusion. Whenever possible, we use unbiased language in our documentation to refer to people of all cultures, ethnicities, genders, and abilities.

© 2026 SAP SE or an SAP affiliate company. All rights reserved.

No part of this publication may be reproduced or transmitted in any form or for any purpose without the express permission of SAP SE or an SAP affiliate company. The information contained herein may be changed without prior notice.

Some software products marketed by SAP SE and its distributors contain proprietary software components of other software vendors. National product specifications may vary.

These materials are provided by SAP SE or an SAP affiliate company for informational purposes only, without representation or warranty of any kind, and SAP or its affiliated companies shall not be liable for errors or omissions with respect to the materials. The only warranties for SAP or SAP affiliate company products and services are those that are set forth in the express warranty statements accompanying such products and services, if any. Nothing herein should be construed as constituting an additional warranty.

SAP and other SAP products and services mentioned herein as well as their respective logos are trademarks or registered trademarks of SAP SE (or an SAP affiliate company) in Germany and other countries. All other product and service names mentioned are the trademarks of their respective companies.

Please see <https://www.sap.com/about/legal/trademark.html> for additional trademark information and notices.

